TrailBuddy Radio Message Spec (Draft v0.5)

23 October 2026 · Spec team · Supersedes Draft v0.4 · Issued together with REQUIREMENTS v0.5 and POLICY v0.5 · Answers OPEN-QUESTIONS.md Q99–Q116 and the 21 findings of the security review of v0.4 (SR-01 to SR-21)

0. How to read this version

Which document wins (Q62). REQUIREMENTS says what the product must do, POLICY says how the clean team works, and this SPEC says exactly how the radio behaves. The three are issued together under one version number. If they disagree, none of them wins: the implementer writes a question in OPEN-QUESTIONS.md and builds nothing that depends on that point until the spec team answers. Until then, for anything affecting SOS, the reading that is safer for the rider applies.

- Section 15 maps every question and every security finding to the section that answers it.
- Decisions made by the product owner are marked **Decision**. Those made by the policy owner are marked **Policy**.
- Section 16 lists every change from v0.4. Section 14 lists what is still open.
- Every rule cites the requirement it serves, in brackets after the section title.
- Nothing here is final until spec v1.0 is frozen.

Main changes in v0.5. The ride frame envelope grows by 4 bytes (a join ID) so that no two radios can share an encryption nonce. Only the rider who raised an SOS can end it, proven by a secret only their radio holds. An SOS repeats at least every 30 seconds. Ride codes expire after 7 days. Pairing, the phone link and updates are hardened. Every test vector is new.

1. How it works

Trail Buddy uses its own radio protocol end to end. The app talks to its own radio over Bluetooth; the radios talk to each other over LoRa. Both conversations are defined here.

```
TrailBuddy app  <-- phone link (Bluetooth) -->  TrailBuddy radio  <-- over the air (LoRa) -->  other TrailBuddy radios
```

Who designs what:

- **Over the air:** this spec defines every frame byte for byte (Sections 4 and 5). The clean team implements these layouts exactly. They are fixed binary layouts, not protobuf.
- **Phone link:** this spec defines every message's fields, types, units and limits (Section 7). The clean team writes the protobuf schema from those tables, following the rules in Section 7.1, and hands the schema file to the product owner for the app team.

Records: hand-overs are recorded in the clean-room log, which the product owner keeps outside the repository. This spec refers only to files in the repository.

Market (Decision): version 1 is for the United States only (REG-001).

2. Foundations

2.1 Radio settings and channel plan (REG-001, Q99)

Decision: all Trail Buddy radios share one common channel. Rides are kept private by their keys, not by separate frequencies.

Proposed settings. These are subject to confirmation by a compliance test lab before any field use:

Setting	Value
Frequency	One fixed channel, centre 916.5 MHz, United States only
Bandwidth	500 kHz
Spreading factor	9
Coding rate	4/5
Preamble	12 symbols
Sync word	0x12
Header	Explicit, with CRC on
Transmit power	22 dBm conducted, on every board
Antenna	Gain at most 3 dBi. Each board profile records the gain of the antenna supplied with that board.
Maximum frame	255 bytes (radio limit); Trail Buddy frames are at most 184 bytes

Antenna (Q99): the 2 dBi in v0.4 was a design figure, not a regulatory limit. The ThinkNode M1's supplied antenna is 2.5 dBi, which is within the new 3 dBi figure, so every board runs at 22 dBm. The lab tests each board with its own supplied antenna. The product owner adds the antenna specifications for the RAK19026 VC and the T114 V2 to `docs/hardware/`.

Regulatory route: the 500 kHz bandwidth is chosen so that the radio can qualify as a digital-modulation device under FCC Part 15 (47 CFR §15.247), which sets no duty-cycle limit. The lab must confirm this. If it can't, the fallback is frequency hopping, which would change Sections 2.1 and 4. Duty-cycle limits would apply only in other regions, which version 1 doesn't serve (**Policy**).

Airtime budget: Trail Buddy's own limit is that normal traffic keeps the channel at or below 30% busy. SOS and SOS replies within the SOS schedule are exempt (4.5). Section 6.10 says what a radio does when the channel is busier than that.

Range trade-off: these settings send each frame quickly, which is what makes 10-second updates possible. Faster settings reach less far than slower ones. Field testing checks range against NFR-001.

2.2 Time (Q75, SR-04)

- **Trail Buddy time** is seconds since 2026-01-01 00:00:00 UTC, as an unsigned 32-bit number. The value 0 means "unknown". It follows UTC, so GPS time must be converted to UTC using the leap-second offset the GPS module reports. A module that already reports UTC needs no conversion.
- **Setting the clock**, in order:
  1. Whenever the radio has a GPS fix, it sets its clock from GPS if the two differ by more than 1 s.
  2. If the radio has had no GPS fix for 10 minutes, it may take the phone's time. Phone time is accepted **only from a bonded phone over an encrypted link**: from SESSION_START and from the fix time of each SHARE_MY_LOCATION (7.4). It is never taken from anything sent before encryption.
     - If the clock is unknown, the radio adopts the phone's time.
     - If the clock is known and differs from the phone's by more than 2 s but no more than 5 minutes, the radio adopts the phone's time.
     - If it differs by more than 5 minutes, the radio keeps its own clock and sets "clock conflict" in STATUS. A GPS fix resolves the conflict.
  3. Otherwise the clock runs freely. The radio's crystal keeps it within 2 s per hour.
- If neither source has set the clock since boot, time is unknown and every time field the radio sends is 0. Items stored while the clock was unknown keep 0; they are not filled in later.
- **The clock may step backwards** when corrected. Intervals, timeouts and ages always use the radio's uptime counter, never the clock, so a step doesn't affect them.
- **No frame is ever dropped because of its time** (SR-04). A fix time or send time more than 60 s ahead of a known clock marks the frame "time suspect": it is still processed, relayed and passed to the app with that flag, and counted (7.4 Diagnostics).
- **Presence** carries a fix age in seconds, not a time, so it works without a clock.
- **The radio never sleeps** (Q113). The processor idles between events, the radio keeps receiving, and the uptime counter counts continuously from boot.

2.3 Identities (Q71, Q72, Q73, SR-01, SR-14, SR-19)

ID	Size	What it is	Where it appears
Radio ID	8 bytes	Random, from the nRF52840 hardware random number generator at first boot. Kept through reboots and firmware updates. A new one is made only by factory reset (6.11).	Phone link after encryption only (RADIO_INFO, rider entries); inside encrypted hello frames. Never sent unencrypted, over the air or over Bluetooth.
Short ID	3 characters	See below.	App display beside names; default display name;

ID	Size	What it is	Where it appears
			Bluetooth name during the pairing window only (7.2).
Rider tag	3 bytes	Ride-scoped, derived from the ride key and the radio ID (4.3). Used for addressing.	Every ride frame envelope; phone link as the rider ID.
Join ID	4 bytes	Random, from the hardware random number generator, chosen each time the radio joins a ride. Stored with the ride record. Makes every radio's nonces distinct (4.4).	Every ride frame envelope.
Anonymous ID	4 bytes	Random, from the hardware random number generator. See 6.6 for when it changes.	Presence frames only.

Short ID:

- Take the first two bytes of the radio ID, most significant bit of the first byte first, and keep the first 15 bits.
- Split them into three 5-bit groups, first group first.
- Write each group as one character from `ABCDEFGHIJKLMNOPQRSTUVWXYZ234567`, where A = 0 and 7 = 31. This is the Base32 alphabet of RFC 4648 §6.
- Radio ID `1122334455667788` gives `CER` (vector in 11.1).

Rider tag rules:

- The value 000000 is reserved for "whole ride". A radio whose derived tag is 000000 moves to the next clash counter.
- **Clash:** a radio that receives a hello carrying its own rider tag and a different radio ID has a clash. Only one of the two radios changes:
  - A radio with an active SOS never changes its tag (SR-10).
  - Otherwise the radio whose radio ID is numerically greater (as an unsigned 64-bit big-endian number) changes. It re-derives its tag with the next clash counter and sends a new hello at once.
  - A radio changes its tag at most once per 10 minutes (SR-19). A further clash within that time is noted and handled when the 10 minutes are up.
  - The clash counter is stored with the ride record and resets to 0 at End Ride.

- Tags are for addressing only. Nonce safety no longer depends on tags being unique (4.4).
- **Merging:** receiving radios key their riders list on the radio ID once a hello has been heard. When a known radio ID appears under a new tag, its entry moves to the new tag, so one rider never appears twice.

Privacy limits, documented (SR-15): during a ride, an outsider can follow a rider tag and join ID for that ride's duration, and can link them to the rider's presence signal, but can't learn the name, radio ID or messages. Because ride codes expire after 7 days and the app makes a new key for every ride (2.4), tags don't carry over from one outing to the next.

2.4 Rides and ride keys (RIDE-001, RIDE-004, RIDE-005, Q72, SR-11, SR-18)

- **Decision:** at most 30 riders per ride. Repeaters count towards the 30.
- A radio holds at most one active ride.
- Every ride frame carries a 2-byte ride tag, so a radio decrypts only frames for its own ride.
- **A 31st radio** with the same QR code still works: radios process and relay its frames, and an SOS from it is handled like any other. The riders list holds 30 entries. RIDE_STATE reports "ride over limit", so the host can be told.
- **New key every ride (Decision):** the app makes a new 16-byte ride key for every ride it creates, using the phone operating system's cryptographic random number generator.
- **Ride codes expire after 7 days (Decision):** the QR code carries an expiry time (8). The radio:
  - refuses JOIN_RIDE with RIDE_EXPIRED if its clock is known and the expiry has passed
  - refuses JOIN_RIDE with BAD_FIELD if the expiry is more than 8 days after a known clock
  - accepts JOIN_RIDE with an unknown clock and checks the expiry once the clock is known
  - when the expiry passes, ends the ride as End Ride does (6.7) and reports RIDE_STATE with RIDE_EXPIRED. During its own SOS it keeps the ride until the SOS ends, so an SOS is never cut off by expiry.
- The app warns riders a day before a ride's code expires.

2.5 Group tiers (RIDE-002)

Decision: four tiers, chosen when the ride is created, carried in the QR code, fixed for the ride.

Tier	Riders	Hop limit	Position interval	Presence while moving	Delivered hop limit	Gap threshold (Fill)
1	2–4	3	10 s	every 10 s	3	60 s
2	5–9	4	45 s	every 10 s	4	135 s
3	10–19	5	120 s	every 20 s	2	360 s
4	20–30	7	90 s	every 20 s	2	270 s

- A radio with no ride sends presence every 10 s while moving.

- Every radio sends presence every 60 s while stopped.

Airtime check (Q66), approximate. It uses the settings in 2.1, the v0.5 frame sizes, and these assumptions:

- relay suppression (4.6) keeps each relayed frame to at most 4 transmissions on average
- one hello per rider every 10 minutes
- one 60-byte text per minute per ride, confirmed by every other rider within Delivered range
- one Fill on Return episode every 10 minutes per ride: an outline and its detail for a 30-minute gap

Tier	Positions	Presence	Hellos	Texts + Delivered	Fill	One ride, total
1 (4 riders)	12.2%	1.8%	0.2%	2.2%	0.7%	17.2%
2 (9 riders)	6.1%	4.1%	0.5%	4.4%	0.7%	15.9%
3 (19 riders)	4.8%	4.3%	1.1%	4.9%	0.7%	15.8%
4 (30 riders)	10.1%	6.8%	1.7%	7.3%	0.7%	26.7%

- **One ride on its own** fits the 30% budget in every tier.
- **Two rides in range plus 10 other Trail Buddy riders**, for example at a trailhead, would go over 30% in every tier (34% to 56%). That is the case Section 6.10 handles: radios slow their own presence first, then their positions, until the channel recovers.
- **An active SOS** adds about 9% in a 30-rider ride (every radio relays each repeat, every 30 s) and about 1% in a 4-rider ride. SOS is exempt from the budget.
- The mesh simulator must confirm these figures, including bursts (build step 5, Q104), and a 4-rider field ride must confirm tier 1 at 10 s.

2.6 Units on the wire

Quantity	Type	Unit	Unknown value
Latitude, longitude	signed 32-bit	millionths of a degree, rounded half away from zero	0x7FFFFFFF for both, only in an SOS with no position (6.4)
Accuracy	unsigned 8-bit	metres, 254 = 254 m or worse	255
Speed	unsigned 8-bit	km/h, rounded to nearest	255
Heading	unsigned 8-bit	1.5° steps (0–239), rounded to nearest step; 360° becomes 0	255; always 255 below 3 km/h
Fix age (presence, SHARE_MY_LOCATION)	unsigned 8-bit	seconds, 254 = 254 s or older	255

Quantity	Type	Unit	Unknown value
Time	unsigned 32-bit	Trail Buddy time (2.2)	0
Signal strength	signed 8-bit RSSI, signed 8-bit SNR	dBm; SNR in quarter-dB steps	RSSI 0
Battery	unsigned 8-bit	percent 0–100	255
Channel busy	unsigned 8-bit	percent 0–100	255
Text and names	UTF-8 (RFC 3629)	length in bytes; no control characters (U+0000–U+001F, U+007F)	—

All names, including the hello name, the display name and the ride name, are 1–24 bytes (Q62). Heading values 240–254 are invalid everywhere.

2.7 Threat model (Decision, SR-05 to SR-18)

Version 1 is designed against these attackers. Where it doesn't fully resist one, the remaining risk is listed as accepted.

Attacker	Version 1 must…	Accepted remaining risk
A stranger with a LoRa radio and no ride key	not be able to read ride traffic, forge ride frames, or stop an SOS from being relayed	They can see presence signals (6.6), link presence to ride traffic during a ride, raise "channel busy", disturb relay suppression for non-SOS traffic, and jam the channel outright (4.6, 6.10).
A stranger with a phone	not be able to pair without someone pressing the radio's button, block the rider's own phone, change the radio's clock, or read the radio's identity	During a pairing window that the rider opened, a stranger's phone could pair first; the rider would notice because their own phone then can't pair (7.2).
A ride member, or anyone who has seen the ride QR code	not be able to end, alter or suppress another rider's SOS, or make radios drop an SOS	They can read the ride's traffic, send texts and positions under any tag, and raise a false SOS under another rider's tag. A false alarm is the safer failure; the impersonated rider's app is told (4.8).
Someone holding the radio	not be able to install unsigned firmware or read keys through the debug port	With laboratory equipment they may extract the current ride key from a captured

Attacker	Version 1 must…	Accepted remaining risk
		radio (POLICY §12). The app's "ride compromised" step (2.4, REQUIREMENTS SEC-005) is the remedy.

3. Product decisions

Topic	Decision
Text to one rider	Not in version 1. Text goes to the whole ride only.
Maximum text	150 bytes of UTF-8. Longer text is refused, not split.
Delivered	The sender sees a count: "Delivered to 3 of 4". The count is defined in 6.3.
Presence default	On by default; always on during the rider's own SOS; back on after End Ride. The app tells the rider at set-up that anyone nearby with LoRa equipment can see their approximate position, speed and heading while presence is on (NEAR-009).
Battery sharing	Not in version 1.
GPS source	Radio GPS first, phone GPS as fallback. The rider can't change this; there is no setting for it (Q61).
Who ends an SOS	Only the rider who raised it, proven by a secret only their radio holds (6.4).
Diagnostics	Counters stay in release firmware; detailed logging is removed.
SOS with no ride (Q79)	An SOS needs a ride in version 1. SEND_SOS with no ride is refused with NO_RIDE, and the app says "Create or join a ride to use SOS".
SOS with no GPS fix (Q80)	An SOS is never held back. See 6.4.
SOS across a reboot (Q81)	An active SOS survives a reboot or battery swap and resumes. See 6.4.
SOS repeat interval	Every 30 seconds at most, until "I'm OK" or End Ride (6.4).
"Left the ride" and "I'm OK"	Each sent at least twice, 10 s apart (6.4).
"Never dropped" (Q59)	The radio's own SOS and own replies are never dropped. Relayed SOS traffic yields to them (4.5).
SOS alerts (Q112)	Shown by the app only: an SOS banner on the Map page and an SOS message with GPS

Topic	Decision
	coordinates on the Messages page. The radio makes no SOS light or sound, because a rider wouldn't see or hear it while riding.
SOS without a phone	Undecided. Today the radio can't raise an SOS on its own (14).
Fill on Return target (Q58)	"Filled" means the outline of the whole gap, up to 30 minutes, reaches the app within 60 s of the rider's return, and the full detail within 3 minutes, while the channel is below 15% busy.
Ride keys and codes	A new key for every ride; codes expire after 7 days (2.4).
Threat model	As in 2.7.
Pairing	Window opens only on a button press (or at first set-up); one new phone per window; already-paired phones are told; no second button press for risky commands (6.11, 7.2).
Updates	Signed, over Bluetooth or USB. Shipped radios have no unsigned route (7.4).
Battery target conditions (Q2)	At least 12 hours with: tier 1, presence on, moving half the time, the board's standard battery, around 20 °C.
Range target (Q2)	On the August field-test route with two riders, total blackout time and the longest blackout must be no worse than that test's result. The longest blackout was 43 minutes.
Version 1 boards	Three boards: the RAK19026 VC base board with the RAK4631 core (lead board, built on first), the Heltec Mesh Node T114 V2 and the Elecrow ThinkNode M1. The WisMesh Pocket V3 is added when it ships. See 9.
"Approaching" definition	Deferred by the product owner. App only; doesn't block firmware.

4. Over the air: frames

All multi-byte numbers are little-endian.

4.1 Ride frame (encrypted) (SR-01)

Offset	Size	Field
0	1	High 4 bits: protocol version (1). Low 4 bits: frame type (1

Offset	Size	Field
		= ride frame).
1	1	High 4 bits: hops remaining (0–7 in version 1). Low 4 bits: priority class (4.5).
2	2	Ride tag
4	3	Sender rider tag
7	4	Sender join ID (2.3)
11	4	Frame counter (4.4)
15	3	Destination rider tag; 000000 = the whole ride
18	n	Encrypted content: 1 kind byte, then the kind's fields (5)
18 + n	8	Authentication tag

Envelope overhead is 26 bytes. The smallest valid ride frame is 27 bytes; the largest is 184 bytes (a 150-byte text).

4.2 Presence frame (not encrypted, never relayed)

Offset	Size	Field
0	1	High 4 bits: protocol version (1). Low 4 bits: frame type (2 = presence).
1	4	Anonymous ID
5	4	Latitude, signed 32-bit in millionths of a degree (as 2.6), rounded to the nearest multiple of 100 (0.0001°, about 11 m), half away from zero
9	4	Longitude, the same unit and rounding
13	1	Heading
14	1	Speed
15	1	Fix age

Exactly 16 bytes. Example: 35.349120° is sent as 35,349,100.

4.3 Keys and derived values (SR-01)

All values are derived from the 16-byte ride key with HKDF-SHA256 (RFC 5869; SHA-256 is FIPS 180-4), no salt (which RFC 5869 defines as 32 zero bytes), using these info labels as ASCII bytes with no terminator:

Value	Length	Info label
Encryption key	16 bytes	`trailbuddy v1 enc`
Ride tag	2 bytes	`trailbuddy v1 ride tag`
Rider tag	3 bytes	`trailbuddy v1 rider tag` followed directly by the radio ID (8 bytes) and the clash counter (1 byte, starts at 0)
Key check value	4 bytes	`trailbuddy v1 check`

Encryption: AES-128 (FIPS 197) in CCM mode (NIST SP 800-38C, RFC 3610), with a 13-byte nonce and an 8-byte authentication tag.

- **Nonce (13 bytes):** sender rider tag (3) + sender join ID (4) + frame counter (4) + ride tag (2).
- **Authenticated, not encrypted:** the 18-byte envelope with the hops-remaining bits set to zero.
- **Encrypted:** the kind byte and its fields.

Policy: these choices are approved by the security review (AES-128-CCM, 8-byte tag, HKDF-SHA256) and are decided for building. Only the security review can change them, and any change comes as a new spec version.

4.4 Nonce uniqueness, frame counter, duplicates and replays (SR-01, SR-02, SR-11, Q70, Q71, Q100)

Requirement (SEC-001): two radios must never use the same nonce under one ride key, and one radio must never use a nonce twice. This holds by design, not by detecting clashes after frames are sent.

How it holds:

- **Between radios:** the nonce contains the sender's rider tag and join ID. For two radios in a 30-rider ride to share both, 56 random-looking bits would have to match: about 1 chance in 2×10^{14} for a 30-rider ride (2^{-47}), well below the 2^{-40} the security review set.
- **Within one radio:** the frame counter never repeats for a given join ID. It starts at 0 when the radio joins a ride (with a new join ID) and only goes up.

The frame counter:

- 32 bits. It increases by 1 for every ride frame the radio creates, never for relays.

- It is stored with the ride record as a **ceiling**: a value at least as high as any counter the radio has used for this join.
- **At boot:** the radio reads both copies of the ceiling and uses the higher valid one. It writes a new ceiling 1024 higher to both copies and reads both back before sending any ride frame.
- **While running:** when the counter comes within 64 of the ceiling, the radio writes and reads back the next ceiling, in both copies, before sending another frame. No frame above a stored, read-back ceiling is ever sent.
- **Missing versus damaged:** a ride record without a valid ceiling counts as damaged, never as "first use".
- **Format:** the location and format of the ride record, including the ceiling, are fixed for every version 1 firmware image, including rollback and recovery images. An image that can't read the ceiling treats it as damaged.
- **Never wraps:** at $2^{32}-1$ the radio stops sending ride frames, reports STORAGE_FAILED, and recovers only by End Ride and joining again (which gives a new join ID). With 7-day ride codes this can't happen in normal use.

If the ceiling can't be stored or read (write failure, both copies damaged, or format unreadable):

- The radio stops sending ride frames, except SOS-class frames: its own SOS and SOS replies (SR-02).
- For those, it chooses a fresh random join ID for this boot only, held in RAM, and counts from 0. Its rider tag is unchanged.
- It sets "storage failed" in STATUS and sends ERROR STORAGE_FAILED.
- It keeps receiving, relaying and sending presence.
- Recovery is End Ride and joining a new ride. The documented remaining risk: if the flash has failed and the radio reboots during an SOS, a fresh join ID could repeat an earlier one with odds of about 1 in 4×10^9 per reboot.

Flash wear budget (SR-02): the storage design must survive, without wearing out any flash page, 10 years of: tier 1 traffic every day, one reboot per minute on average, and every counter, ceiling, event-number and SOS-state write in this spec. The clean team records the design and the calculation in DECISIONS.md.

Duplicate memory: a frame's identity is (sender rider tag, sender join ID, frame counter). The radio remembers up to 256 identities; each expires after 10 minutes, and the oldest is evicted when full.

- Its own frames are entered too, so their echoes are never relayed.
- A duplicate is counted for relay suppression (4.6) before it is dropped.

Replay window: for each sender (rider tag and join ID), the radio keeps the highest counter seen and a 64-frame window below it, for up to 32 senders, evicting the least recently heard. Frames below the window, or already marked in it, are dropped. Neither the window nor the duplicate memory is kept across reboot.

Replay after a reboot (SR-11): an old recorded frame can be accepted once by a radio that has just rebooted. To limit the effect:

- Texts and SOS replies carry a send time (5). When both the sender's and the receiver's clocks are known and the send time is more than 10 minutes older than the receiver's clock, the frame is marked "old": the app shows it as old, and an old text creates no Delivered.
- An SOS carries its first-sent time, and the app always shows how long ago an SOS was first sent. An SOS is never discarded for being old.
- Ride codes expire after 7 days (2.4), so traffic recorded from an earlier ride is never valid on a later one.

4.5 Priority classes and queues (Q59, Q69, SR-06)

Class	Kinds	Queue length (own / relay)	When full
0	SOS, SOS reply	16 own / 48 relay	See below
1	Presence	1 own	The newest replaces the queued one
2	Position	1 per rider tag, own and relayed, up to 31	The newest position of a rider replaces that rider's queued one
3	Delivered	4 / 8	Oldest relayed entry dropped first, then oldest own
4	Hello, hello request	2 / 4	A newer hello from the same sender replaces the queued one; otherwise oldest relayed dropped
5	Text	4 / 8	A relayed text is dropped. An own text is refused with BUSY.
6	Fill ask, fill answer	4 / 8	Oldest relayed entry dropped first, then oldest own

- Own frames and relayed frames share each queue, but each class keeps part of its space for the radio's own frames, as shown.
- **Class 0:**
  - The radio's own SOS and own SOS replies always have queue space and are always sent before relayed class-0 frames. They are never dropped.
  - A newer repeat of the same alert replaces the queued older one, own or relayed.
  - Replies are keyed on alert owner, alert ID, sender and reply value, so two riders' "Copy that" never replace each other.

  - When the relay part is full, a relayed entry is dropped to make room: the oldest relayed "Copy that" first, then the oldest relayed entry of any kind.
- **SOS schedule limit (SR-06):** class-0 treatment for relays (own queue part, no suppression, no airtime budget, bounded channel wait) applies only to traffic consistent with the SOS schedule. For each originating sender (rider tag and join ID), the radio treats as class 0 at most:
  - one active alert ID at a time (a new alert ID replaces the previous one)
  - 3 SOS frames in any 30 s
  - 6 SOS replies in any 60 s

  Frames over these limits are not dropped. They are relayed as class 6 frames, with normal suppression, budget and channel rules, and counted ("SOS over schedule").
- A relay cancelled by suppression leaves its queue.
- Every drop because a queue is full is counted (DIAGNOSTICS, "queue full").
- Lower class numbers always go first. A transmission already in progress is never cut off. The longest frame lasts about 240 ms on air.

4.6 Relaying (MESH-001 to MESH-003, Q68, SR-09, SR-17)

A radio relays a ride frame only if all of these hold:

- it carries the radio's own ride tag
- it passes every receive check (4.8)
- it isn't a duplicate
- hops remaining is above 0
- it isn't addressed only to this radio

Radios don't relay other rides' frames. Presence frames are never relayed.

Delay before relaying:

- **SOS and SOS replies within the schedule limit (4.5):** uniform random 20–500 ms, then the bounded channel check in 4.7. Always relayed; never suppressed. The wider window, compared with v0.4, reduces collisions between neighbours relaying the same SOS.
- **Everything else:**
  - Delay = 20 ms + s × 1000 ms × u + 200 ms × v, where u and v are independent uniform random numbers from 0 to 1.
  - s is set by the SNR of the first copy received, in dB (the wire value divided by 4): s = (SNR + 10) / 20, clamped to 0.1–1.0.
  - So frames heard weakly relay sooner than frames heard strongly.
- **Suppression (not for SOS):** while waiting, each further reception of the same identity counts one, whatever its hops remaining; the first copy received doesn't count. At 2, the radio cancels its relay.
- **Hops can only grow for SOS (SR-17):** for SOS and SOS replies, if a radio that hasn't yet relayed a frame hears a further copy with more hops remaining than the copy it holds, it adopts the higher value. If it had decided not to relay because the first copy had 0 hops

remaining, it now relays once. So a copy with its hops tampered down can't stop an SOS spreading.

- The relayed frame is identical except that hops remaining is reduced by 1. Because those bits aren't authenticated, a receiver drops any frame whose hops remaining is above that kind's starting value.

This combines the distance-based and counter-based schemes from Ni, Tseng, Chen and Sheu, "The Broadcast Storm Problem in a Mobile Ad Hoc Network", MobiCom 1999.

Per-kind rules:

Kind	Destination	Starting hops	Class	Acknowledged
Position	Whole ride	Tier hop limit	2	No
Hello	Whole ride	Tier hop limit	4	No
Hello request	Whole ride	Tier hop limit	4	Answered by hellos
Text	Whole ride	Tier hop limit	5	By Delivered
Delivered	Original sender	Delivered hop limit (2.5)	3	No
SOS	Whole ride	7	0	By SOS replies
SOS reply	Whole ride	7	0	No
Fill ask	Whole ride	Tier hop limit	6	By fill answer
Fill answer	Whole ride	Tier hop limit	6	No
Presence	Any Trail Buddy radio	Not relayed	1	No

A frame whose class field differs from its kind's class in this table is malformed (4.8), except a relayed SOS-class frame that a radio has moved to class 6 under the schedule limit (4.5). That change is made only in the radio's own queue; the frame sent keeps its original class bits.

4.7 Sending a frame: listen before talking (MESH-005, Q64, SR-09)

Every transmission follows these steps. A relay goes through them after its relay delay.

1. Take the queued frame with the lowest class number; within class 0, own frames first; otherwise the oldest. A frame that arrives in a lower class before the transmission starts goes first.
2. **Channel check:** run the radio's channel activity detection (SX1262 CAD) once.
3. **Clear:** transmit at once.
4. **Busy, normal frame:**
   - Wait a uniform random time from 0 to 50 ms × 2^k, with k starting at 1 and rising by 1 after each busy check, up to k = 5 (up to 1.6 s).
   - Then go back to step 1.

   - After 8 busy checks for the same frame, drop it and count it under "gave up".
5. **Busy, SOS or SOS reply within the schedule limit:** wait a uniform random 20–100 ms and check again. Keep doing so until the channel is clear or 1 s has passed since the frame was queued for sending; then transmit whether or not the channel is clear. So an SOS listens before talking but is never delayed by channel access for more than 1 s.
6. After transmitting, the radio returns to receiving at once.

SOS delivery target (SOS-006): the mesh simulator must show that at tier 4, with 30 radios in the densest layout used for testing, an SOS reaches every radio within the hop limit within 10 s of its first transmission with probability at least 0.95, and within 40 s (by the repeats at 10 s and 30 s) with probability at least 0.999.

4.8 Receive checks (Q67, SR-03, SR-06, SR-13)

Any frame that fails a check is dropped, not relayed, and counted under the reason shown. **General rule: any field value not listed as valid in this spec is invalid.** Checks run in this order:

#	Check	Drop reason
1	Protocol version is 1	Unknown version
2	Frame type is 1 or 2	Malformed
3	Length: presence exactly 16; ride frame 27–184 bytes. Checked before any decryption.	Malformed
4	Ride frame: ride tag is this radio's ride	Wrong ride (no further checks)
5	Sender rider tag is not 000000	Malformed
6	Authentication passes	Failed authentication
7	Not sent as this radio: if the sender rider tag and join ID are this radio's own, the frame must be one this radio sent (an echo in its duplicate memory)	Sent as you (reported to the app, see below)
8	Not a duplicate, and inside the replay window	Duplicate
9	Kind is 0x01–0x09	Malformed
10	Priority class matches the kind (4.6 table)	Malformed
11	Content length is exactly what the kind's fields require, including length	Malformed

#	Check	Drop reason
	bytes and counts: fill points 0–11, rider count 1–8, SOS reply 10 bytes for Copy that and 26 bytes for I'm OK and left the ride	
12	Hops remaining is not above the kind's starting hops	Malformed
13	Enumerated and ranged values are valid: position source 0–1; SOS reply 1–3; fill part number below total parts; total parts at least 1; range from no later than range to; accuracy any value	Malformed
14	Latitude within ±90°, longitude within ±180°, except the SOS no-position marker	Malformed
15	Fix time not 0, except an SOS with no position. A time ahead of the clock never drops a frame (2.2).	Malformed
16	Reserved bits are 0 (hello flags bits 1–7; fill part byte as defined)	Malformed
17	Text and names are valid UTF-8 with no control characters, 1 byte or longer	Malformed
18	Destination is the whole ride or this radio's tag, or the frame is to be relayed	Not an error; addressed to another rider

- **Sent as you (check 7, SR-03):** the frame is dropped and counted, and STATUS "frames sent as you" increases, so the app can tell the rider someone is sending under their identity. A radio never ends or changes its own SOS because of a received frame.
- **Presence frames** go through checks 1–3, the latitude and longitude ranges, and heading 0–239 or 255, then the plausibility rules in 6.6.
- **Fuzz testing:** these checks are the basis for the fuzz tests in POLICY §18.

4.9 Key handling (SEC-005, SR-18)

- Ride keys come from the phone operating system's cryptographic random number generator. Radio IDs, join IDs, anonymous IDs and SOS secrets come from the nRF52840 hardware random number generator.

- No key, derived key or SOS secret is ever logged, put in diagnostics, or sent back over the phone link.
- End Ride, ride expiry and factory reset erase key material from every flash copy (both copies of the ride record, and any old copies left in flash pages), not just mark it deleted.
- Release builds lock the processor's debug port (POLICY §12).

5. Over the air: kinds inside a ride frame

The first encrypted byte is the kind.

Kind	Byte	Fields (in order)	Size after kind byte
Position	0x01	latitude (4), longitude (4), accuracy (1), fix time (4), source (1: 0 = radio GPS, 1 = phone GPS)	14
Hello	0x02	radio ID (8), anonymous ID (4), flags (1: bit 0 = repeater; bits 1–7 = 0), name length (1, 1–24), name	15–38
Hello request	0x03	none	0
Text	0x04	message ID (2), send time (4), text length (1, 1–150), text	8–157
Delivered	0x05	message ID (2), original sender rider tag (3)	5
SOS	0x06	alert ID (2), latitude (4), longitude (4), accuracy (1), fix time (4), time first sent (4), repeat number (2, stops at 65535), owner commitment (8)	29
SOS reply	0x07	alert ID (2), alert owner rider tag (3), reply (1: 1 = copy that, 2 = I'm OK, 3 = ended because the owner left the ride), send time (4), then	10 or 26

Kind	Byte	Fields (in order)	Size after kind byte
		for replies 2 and 3 only: owner secret (16)	
Fill ask	0x08	rider count (1, 1–8), then per rider: rider tag (3), from time (4), to time (4)	12–89
Fill answer	0x09	rider tag (3), range from (4), range to (4), part (1: bit 7 = outline, bits 0–6 = part number from 0), total parts in this pass (1), point count (1, 0–11), then per point: latitude (4), longitude (4), fix time (4)	14–146

SOS owner proof (SR-03):

- When an alert starts, the owner's radio draws a random 16-byte **owner secret** and stores it with the SOS state (6.12).
- Every SOS frame for that alert carries the **owner commitment**: the first 8 bytes of SHA-256 of the ASCII label `trailbuddy v1 sos` followed by the owner secret.
- An "I'm OK" or "left the ride" reply carries the owner secret. A receiver ends the alert only if SHA-256 of the label and the carried secret gives the commitment it holds for that alert. Otherwise the reply is ignored and counted ("bad SOS proof").
- A receiver that has heard a reply ending an alert but no SOS frame for it (so it holds no commitment) stores the reply and applies it if a matching SOS frame arrives within 24 hours.
- "Copy that" carries no secret; anyone in the ride may send it.

Message IDs and alert IDs (Q74):

- 16-bit counters kept by the sending radio and stored in flash, written 16 ahead like the frame counter, so they survive reboots.
- They start at 1, skip 0, and wrap from 65535 to 1.
- A received SOS whose owner and alert ID match a known alert but whose first-sent time differs is a new alert (SR-10).

6. Behaviour

6.1 Position (POS-001 to POS-003, Q57, Q77)

- **Schedule:** the radio sends a position at the tier interval, with up to 10% random jitter added.
- **Source (Decision: radio first):**
  1. The radio's own GPS fix, if it is no older than 2 × the tier interval.
  2. Otherwise the phone's latest position, if it is no older than 2 × the tier interval.
  3. Otherwise the newer of the two, if it is no older than 10 minutes.
  4. Otherwise nothing is sent, and STATUS reports "no position".

  The position always carries its true fix time. Receivers judge freshness by it.
- **Age of a phone fix (Q116):** SHARE_MY_LOCATION carries the fix's age when the app sent it. The radio adds the time since the message arrived, measured with the uptime counter. This works with an unknown clock and isn't fooled by a delayed message.
- The app sends SHARE_MY_LOCATION whenever the phone has a new fix, at most once per second. There is no reply.
- After the phone disconnects, the radio keeps sending positions from its own GPS.
- **Moving or stopped:**
  - Moving means 5 km/h or more for 10 s. Stopped means below 3 km/h for 30 s.
  - Between those speeds the previous state holds. At boot, and whenever speed is unknown, the state is "moving".
  - GPS duty and presence use the same state.
- **GPS duty (POS-003):** the GPS runs continuously while moving or while presence is on. When the rider has been stopped for 5 minutes with presence off, it samples once per minute. A stopped rider keeps sending positions at the tier interval, using the latest sample (rule 3 above), so they never go silent and never trigger Fill on Return.

6.2 Rider hello (MSG-003, Q76, SR-19)

- A radio sends a hello:
  - when it joins a ride
  - when its name changes
  - when its rider tag changes
  - when its anonymous ID changes
  - every 10 minutes
  - in reply to a hello request, at most once per minute however many requests it hears
- A radio that hears a frame from a rider tag it has no hello for sends one hello request, at most once per minute. Each radio that hears a request replies after a random delay of 0 to (5 + 0.2 × riders in contact) seconds, so replies spread out in larger rides (Q104).
- **Display name:** 1–24 bytes of UTF-8 with no control characters. The default is "Rider" plus the short ID. Two riders may share a name; the app shows the short ID beside duplicates.

6.3 Messages (MSG-001, MSG-002, Q60, Q74, Q86, Q103, Q104, SR-20)

In contact (one definition, used everywhere): a ride member is in contact if any authenticated frame from them has been heard within 3 × the tier's position interval, with a minimum of 60 s. STATUS ride contact, RIDERS_LIST "in contact" and the Delivered total all use it.

Texts:

- Text goes to the whole ride, 150 bytes maximum. It carries its send time (0 if the sender's clock is unknown).
- Every rider radio that receives a text stores it and sends Delivered to the sender, whether or not its phone is connected, after a random delay of 0 to (2 + 0.1 × riders in contact) seconds (Q104). Repeaters don't send Delivered. A text marked "old" (4.4) gets no Delivered.
- **The Delivered count** ("Delivered to 3 of 4"):
  - The first number is distinct riders who confirmed.
  - The second is the ride members in contact when the text was first sent **and within Delivered range**, not counting the sender or repeaters (Q103). A member is within Delivered range if their latest frame reached this radio having travelled no more hops than the tier's Delivered hop limit (2.5).
  - A rider's confirmation counts once, however many arrive.
- **Sender's side:**
  - SEND_RESULT reports "queued", then "sent", then "delivered" with the count each time a new rider confirms.
  - If **no** rider has confirmed after 20 s, the radio resends the text with the same message ID and a new frame counter, up to 3 times. After the third resend with none, SEND_RESULT reports "failed", reason NO_CONFIRMATION. A text with at least one confirmation is never resent (Q103).
  - A Delivered that gives up (4.7) is not retried; the count simply stays lower.
- **Receiver's side:**
  - A radio remembers (sender rider tag, message ID) for 30 minutes.
  - A resent text it already holds creates no new event and isn't passed to the app again, but it sends Delivered again, because the first may have been lost.

Stored events:

- The radio numbers everything it stores for the app with a 32-bit event number: texts, SOS events, SOS replies and rider changes. Event numbers only go up; the next number is stored in flash, like the counters in 5, so it survives reboots and End Ride. Factory reset restarts it.
- **Storage:**
  - The radio keeps the last 200 events in memory.
  - SOS events are also kept in flash, and stay until that SOS has ended and a further 24 hours have passed.
  - An SOS's repeats update its existing event rather than creating new ones.

- **An event counts as sent** once it has been written to a connected app.
- **GET_MISSED:**
  - The app sends the last event number it holds.
  - The radio answers with EVENTS_INFO first, giving the oldest event number it still holds and the newest. If the oldest is above the app's number plus 1, events were lost and the app can say so.
  - It then replays the events after the app's number as their normal messages, in order, each carrying the request ID, with the last one flagged.
- **App rule (SR-20):** the app starts its event count again from 0 whenever RADIO_INFO shows a different radio ID from last time, or EVENTS_INFO's newest number is below the app's own.
- **Events waiting** in STATUS counts stored events not yet sent to the app.

6.4 SOS (SOS-001 to SOS-006, Q79–Q83, Q90, Q112, SR-03, SR-10)

- **Needs a ride (Decision):** SEND_SOS with no active ride is refused with NO_RIDE.
- SEND_SOS starts an alert. The radio assigns the alert ID and draws the owner secret (5). The radio's reply is SOS_STATUS with the request ID.
- **Position (Decision: an SOS is never held back):**
  - The SOS goes at once with the best position available: the source rules in 6.1 first. If those give nothing, the last known position of any age, with its true fix time, so the app can show how old it is.
  - If the radio has never had a position since it was switched on, the SOS goes with the "no position" marker: latitude and longitude 0x7FFFFFFF, accuracy 255, fix time 0.
  - Each repeat carries the latest position.
- **Schedule (Decision: at most 30 seconds between sends):**
  - The first send is at time t0.
  - Then t0 + 10 s.
  - Then every 25 s plus a random 0–5 s (never subtracted), measured from the previous send, so the gap is never more than 30 s.
  - This continues until the alert ends. A missed slot is not caught up.
- **Repeat number:** 0 on the first send, then +1 for each repeat. It stops at 65535; it doesn't start again from 0.
- **Survives reboot (Decision):**
  - The radio stores the alert ID, owner secret, first-sent time, repeat number and latest position in flash when the SOS starts and at least once a minute while it is active.
  - After a reboot or battery swap, it sends a repeat as soon as it is running, then carries on.
- A second SEND_SOS while an alert is active updates its position only; there is one active SOS per radio.
- The radio keeps repeating whether or not the phone is connected.
- **I'm OK:** only the owner's app can send it for its own radio's alert (SOS_REPLY for someone else's alert with IM_OK is refused with NOT_ALLOWED_NOW). The radio sends

it twice, 10 s apart, carrying the owner secret. Afterwards, while still in the ride and for 24 hours, whenever it hears a Copy that or a relayed SOS frame for that ended alert, it sends I'm OK again, at most once per 30 s (SR-10).

- **Copy that:** informational only; it doesn't stop repeats.
- **Ending:**
  - An SOS ends when the owner sends I'm OK, or when the owner ends the ride.
  - **End Ride during the owner's own SOS (Decision):** the radio sends the "owner left the ride" reply, with the owner secret, twice, 10 s apart. It keeps the ride key until both copies have been sent, or 60 s have passed. Then it erases the ride and reports RIDE_STATE (not active).
  - **Factory reset during the owner's own SOS:** the radio first sends "left the ride" twice, as for End Ride, then resets.
  - **Damaged SOS record at boot:** the radio reports SOS_RESET (6.9). The app tells the rider the SOS was lost and must be raised again.
  - A radio never ends its own alert because of a received frame.
  - Radios that never hear a valid end keep showing the SOS with its last-heard time. It is never ended automatically for lack of signal.
- **SOS_STATUS** (the owner's own alert) is sent:
  - in reply to SEND_SOS
  - after every send and repeat
  - after each new Copy that
  - when the alert ends

  Its Copy-that list holds the distinct riders who replied Copy that to this alert, and clears when the alert ends. "Last sent" is 0 when the clock is unknown.
- **To the app:**
  - SOS_RECEIVED is sent for a new alert and for every repeat, with the alert's original event number; the app de-duplicates by alert. It carries the first-sent time; the app always shows how long ago the SOS was first sent.
  - SOS_ENDED is sent when a valid end is heard.
- **Alert on the radio (Q90, Q112, Decision):** none. The radio makes no SOS light or sound, because a rider wouldn't see or hear it while riding. The rider is alerted by the app (SOS-005): an SOS banner on the Map page and an SOS message with GPS coordinates on the Messages page, both driven by SOS_RECEIVED.

6.5 Fill on Return (FILL-001, Q58, Q84, Q38, Q104, Q108)

Storage: each radio keeps its own position history for the last 2 hours, one point every 10 s, and the positions it has heard from ride members (6.12).

Definitions:

- **Return:** the moment the first authenticated frame, direct or relayed, is heard from a ride member after a silence longer than the tier's gap threshold.

- **Gap:** from the fix time of that rider's last position before the silence to the fix time of their first position after it. If that is more than 30 minutes, the gap is the latest 30 minutes before the return.
- **Filled (Decision):** the outline of the whole gap has reached the app within 60 s of the return, and the full detail within 3 minutes, while the channel is below 15% busy.

The rules:

1. **Gap:** a ride member returns, and has moved more than 50 m, measured between their last position before the silence and their first position after it.
2. **Ask:**
   - Wait a random 2–8 s.
   - Send one fill ask listing every rider who returned, up to 8 per ask, each with their gap.
   - Leave out any rider for whom another radio's ask with an overlapping gap was heard during the wait.
   - A radio whose clock is unknown waits until it is known before asking.
3. **Answer:**
   - Only the returning rider's own radio answers, from its own history, for its own tag. This works whether or not its phone is connected.
   - It answers each asked gap in one pass:
     - First the **outline:** one point per 60 s of the gap, up to 30 points, in up to 3 parts.
     - Then the **detail:** every stored point in the gap, at 10 s spacing, in up to 17 parts. The detail repeats the outline's points, so receivers simply drop points they already hold.
   - Each part names the range it covers, its part number and the total parts in its pass (outline or detail), so receivers know when a pass is complete.
   - **No history (Q108):** if the radio holds no points for an asked range (for example because it rebooted during the gap), it sends one fill answer with a point count of 0, outline bit clear, part 0 of 1, covering the asked range. Receivers mark that gap as answered and don't ask again until the rider's next return.
   - The answer goes to the whole ride, so every radio with the same gap uses it.
   - An ask whose gap was fully covered by an answer this radio sent in the last 15 s is skipped.
4. **Retry:**
   - A gap is **answered** when every part of the detail pass covering it has been received, or an empty answer has.
   - If any part is missing 30 s after the last part was heard, or after the ask if nothing came, the radio asks again for just the missing sub-range.
   - After 4 asks for the same gap, it waits for the rider's next return.
5. **Channel:**
   - Fill frames are sent only on a channel-busy reading under 60 s old (6.10).
   - The rate depends on that reading:

        - below 15%: up to 1 fill frame every 3 s
        - from 15% to 25%: 1 every 10 s
        - above 25%: outline parts only, at 1 frame every 30 s; detail waits until the channel is below 25% (Q104)
    - At slow-down step 3 (6.10), fill stops entirely.
    - Any queued SOS frame blocks fill. Blocked frames wait and go when the block ends.

Timing:

- At one frame every 3 s, a 30-minute gap's outline (3 parts) takes about 10 s, and its detail (17 parts) about 50 s more. That meets the 60 s and 3-minute targets with a margin for the ask and relays.
- At 1 every 10 s, the outline takes about 30 s and the detail about 3 minutes.
- On a channel above 25% busy, the outline takes about 90 s. The "filled" target doesn't apply there.

To the app:

- Received fill points are added to the radio's stored history of that rider. Points already held, same rider and fix time, are dropped.
- New points reach the app as TRACK_POINTS with "filled" set, sorted by fix time, outline first.
- GET_TRACK returns stored history only; it doesn't cause a fill ask.

6.6 Nearby riders (NEAR-001 to NEAR-009, Q76, Q78, Q47, Q116, SR-15, SR-16)

- **Sending:**
    - Presence goes at the interval in 2.5, with ±1 s of random jitter.
    - It uses the radio's GPS fix if it is no older than 10 s, otherwise the phone's latest fix if it is no older than 10 s (age as in 6.1). Otherwise no presence is sent.
    - The fix age is measured with the uptime counter, so it works with an unknown clock.
    - Heading and speed come from the same source; 255 when unknown.
- **Not relayed:** a presence frame travels one hop only.
- **Anonymous ID changes:**
    - A new one is made at every boot, and when 6 hours of uptime have passed since the last change.
    - It is applied while the rider is stopped, so one encounter doesn't show a rider twice. But it is always applied within 24 hours of the last change, even while moving.
    - The radio sends a hello as soon as its ID changes, so its ride mates can keep hiding it.
    - **What this protects (SR-15):** the change stops casual linking of a rider from one day to the next. It doesn't stop someone nearby tracking a rider during a ride. The app says so at set-up (NEAR-009).

- **What reaches the app (NEAR-004):** the radio passes to the app the presence frames it receives, except:
  - its own
  - those from members of its own ride, recognised by the anonymous IDs in their hellos
  - reports that fail the plausibility check
  - repeat reports from the same anonymous ID within 5 s
- **Nearby table (SR-16):**
  - The radio tracks up to 32 anonymous IDs. An entry expires 5 minutes after its last accepted report.
  - An ID is **confirmed** once it has 2 or more plausible reports.
  - When the table is full, a new ID may take the place only of an unconfirmed entry, or of an entry with no accepted report in the last 60 s, least recently heard first. A confirmed entry heard in the last 60 s is never pushed out.
  - A new ID that finds no place is not passed to the app. The radio counts it ("nearby overflow") and sets "nearby view incomplete" in STATUS for 5 minutes, so the app can say the view may be missing riders (NEAR-008).
  - The radio sends at most 4 NEARBY_RIDER messages per second in total. Any more are not sent; the app can ask for NEARBY_LIST.
- **Plausibility:**
  - The radio compares each report with the last accepted report from that ID. The distance between them is divided by the time between receipts, adjusted by the difference in their fix ages.
  - Above 200 km/h, the report is rejected and doesn't replace the last accepted one.
  - The first report from an ID is always accepted, subject to the table rules above.
  - The remaining risk of fake presence is accepted and documented (2.7).
- **Turning presence off** stops sending only; receiving is always on. Presence is forced on during the rider's own SOS, and on again after End Ride.
- **Alerts:** approaching-rider warnings are made by the app only in version 1.
- **Combining with position:** presence and ride position stay separate signals in version 1.

6.7 End Ride (RIDE-003, Q83, Q87)

Item	After End Ride
Ride key, name, tier, expiry, tags, clash counter, join ID, frame counter ceiling	Erased from every flash copy (4.9), after any "left the ride" replies have been sent (6.4)
Own active SOS	Ended, with the "left the ride" reply sent twice first
Queued ride frames	Dropped
Stored texts, ride events, track history, known riders	Erased (SOS events stay in flash for 24 hours)
Display name	Kept
Presence sending	On

Item	After End Ride
Repeater mode	Off
Message and alert ID counters, event number	Kept
Diagnostics counters	Kept
Paired phones	Kept

If any write fails, RIDE_STATE carries the error code STORAGE_FAILED. End Ride affects only the local radio. Ride expiry (2.4) has the same effect.

6.8 Repeater mode (RPT-001, Q89)

- **Set-up:** the phone joins the radio to a ride, then turns repeater mode on. CHANGE_SETTINGS returns NO_RIDE with no ride, and SOS_ACTIVE during the rider's own SOS. The radio keeps the ride and the mode through reboots until the phone ends the ride, turns repeater mode off, or the ride code expires.

A repeater…	
Relays its ride's frames	Yes
Sends a hello every 10 minutes, with the repeater flag set, and answers hello requests	Yes
Sends positions or presence	No
Sends Delivered	No; it isn't counted in Delivered totals or ride contact
Counts towards the 30-rider limit and appears in RIDERS_LIST, flagged	Yes
Answers fill asks	No; it has no position history. It relays fill frames.
Keeps stored events and passes nearby presence to a connected phone	Yes
Accepts already-paired phones, and opens the pairing window only on a button press, like any radio	Yes

REPEATER_STATUS is sent together with STATUS whenever repeater mode is on. Because a repeater is often left unattended, the pairing rules in 6.11 matter most for it (SR-07).

6.9 Settings rules and damaged records (Q87, Q88, Q102, Q110)

- A change is written all-or-nothing. If any setting ID is unknown or any value is invalid, nothing changes and the reply is BAD_FIELD. Text values follow the same rules as over the air: valid UTF-8, no control characters, 1–24 bytes (Q110).
- A value equal to the stored one is not rewritten.
- SETTINGS_RESULT always returns every setting as stored.
- **Presence turned off during the rider's own SOS:** stored, but it takes effect only when the SOS ends. During the SOS, SETTINGS shows the stored value and STATUS shows presence sending as on.

- Each stored record (6.12) has two copies and a CRC.

Record damaged in both copies at boot	What the radio does	Reported to the app (after encryption, Q102)
Settings	Resets them to defaults; repeater mode off	ERROR SETTINGS_RESET
Ride	Erases the ride	ERROR RIDE_RESET, then RIDE_STATE not active
Active SOS	The SOS ends locally; it can't be resumed	ERROR SOS_RESET; the app tells the rider to raise the SOS again
Frame counter ceiling	Stops sending ride frames except SOS-class (4.4)	ERROR STORAGE_FAILED
Paired phones	Forgets all phones; the pairing window opens as at first set-up	Nothing can be sent until a phone pairs again; the app shows the "forget this device" help (7.2)

6.10 Channel busy and the airtime budget (MESH-004, Q65, Q66, SR-17)

Measuring:

- Every second the radio adds up the time it spent:
  - transmitting
  - receiving any LoRa frame with a valid header on the channel, from any ride, presence, or otherwise, worked out from the frame's length with the standard LoRa time-on-air formula
  - 10 ms for each channel check that found activity without a frame following
- **Channel busy** is that total over the last 60 s, as a percentage, updated every second.
- For the first 60 s after boot, it is computed over the time so far. During the first 10 s it is reported as 255 (unknown).

Above the budget (runtime rule):

- If channel busy stays above 30% for 60 s, the radio slows its own traffic in steps. It takes the next step after another 60 s if the channel is still above 30%:
  1. presence interval doubled
  2. position interval × 1.5
  3. fill frames stopped
- When busy stays below 20% for 60 s, it undoes one step at a time, in reverse order.
- SOS and SOS replies within the schedule limit (4.5) are never slowed.
- STATUS reports the current step, 0–3, so the app can say updates are slower because the channel is busy.
- **Documented remaining risk (SR-17):** anyone transmitting on the channel raises channel busy, so an outsider can force radios to slow down. The steps are limited to 3 and SOS is exempt.

6.11 Button, pairing window and factory reset (Q71, Q94, SR-07)

Every version 1 board has one **user button**, named for each board in Section 9.

Action	Result
Press the button for 2 s while running	Opens the pairing window for 2 minutes
The radio starts with no phone paired (first set-up, or after "forget phones" or factory reset)	Pairing window opens for 2 minutes
The radio starts with at least one phone paired	No pairing window
Hold the button for 10 s while running	Forgets all paired phones
Hold the button while the radio starts (power-on or reset), for 10 s	Factory reset

The pairing window (Decision):

- It opens only as above. It never opens because of a reboot, crash, battery swap or update when a phone is already paired.
- It closes as soon as one new phone pairs, or after 2 minutes, whichever comes first. So one window lets exactly one new phone in.
- If 4 phones are already paired, the window doesn't open, and a new phone is refused (7.2).
- The status light shows the window and a new pairing (9).
- There is no second button press for risky commands from a newly paired phone (**Decision**).

Factory reset erases:

- the radio ID, and makes a new one
- settings
- the ride, from every flash copy (after "left the ride" replies if an SOS is active, 6.4)
- paired phones
- stored events and history
- the message and alert ID counters
- the event number

A new radio ID gives new rider tags, and the next join gives a new join ID, so no nonce can repeat (4.4).

6.12 What is stored, and what survives a reboot (Q85, SR-02)

Record	Where	Survives reboot	Capacity and eviction	Erased by End Ride
Radio ID	Flash	Yes	1	No (factory reset only)

Record	Where	Survives reboot	Capacity and eviction	Erased by End Ride
Settings	Flash, 2 copies + CRC	Yes	1	No
Ride (key, name, tier, expiry, clash counter, join ID, frame counter ceiling)	Flash, 2 copies + CRC; fixed format for all version 1 images	Yes	1	Yes, from every copy
Message ID, alert ID, event number counters	Flash, written 16 ahead	Yes	1 each	No
Active SOS state (alert ID, owner secret, first-sent time, repeat number, latest position)	Flash, 2 copies + CRC	Yes	1	Yes, after the “left the ride” replies
Ended own alerts (alert ID, owner secret), for re-sending I’m OK	Flash	Yes	Last 4, for 24 hours	Yes
SOS events	Flash	Yes	All, until 24 h after the SOS ends	No
Paired phones	Flash, 2 copies + CRC	Yes, and through firmware updates	4; a fifth is refused	No
Own position history	RAM	No	720 points (2 h at 10 s)	Yes
Heard position history	RAM	No	4096 points shared; when full, points older than 30 minutes are thinned to one per 30 s, then the oldest are dropped	Yes
Other stored events	RAM	No	200	Yes
Known riders	RAM	No; rebuilt from	30 (+1 over-limit	Yes

Record	Where	Survives reboot	Capacity and eviction	Erased by End Ride
(names, radio IDs, anonymous IDs)		hellos and hello requests	entry)	
Pending fill asks	RAM	No	8	Yes
Duplicate memory, replay windows	RAM	No	4.4	No
Nearby table	RAM	No	32	No
Diagnostics counters	RAM	No	7.4	No

7. Phone link

7.1 Schema rules (Q5, Q92, Q105)

- protobuf (Google's proto3 language specification), package `trailbuddy.v1`, written by the clean team from the tables below. The clean team hands the `.proto` file to the product owner for the app team.
- **Names:** message names in CamelCase from this spec (HELLO becomes `Hello`, SEND_TEXT becomes `SendText`); field names in snake_case.
- **One wrapper message,** `Envelope`, carries every message:
  - `request_id` (field 1)
  - `last` (field 2)
  - a `oneof body` with one field per message type, numbered from 10 in the order the messages appear in Section 7.4
- **Field numbering:** in table order, starting at 1. Numbers are never reused; removed fields are reserved.
- **Presence tracking:** every scalar field is marked `optional`, so the receiver can tell "missing" from 0. The receiver checks the R and O markings in 7.4 and answers a missing required field with BAD_FIELD.
- **Types (Q105):** latitude and longitude use `sfixed32`; times use `fixed32`; radio IDs, keys, hashes and signatures use `bytes`; other numbers use the smallest unsigned or signed varint type that holds their range. Fixed-width types keep sizes predictable.
- Every enum starts with an "unspecified" value of 0.
- Every text field and list is bounded by the limits given here.
- **Sizes:** the clean team records the worst-case encoded size of every message in DECISIONS.md, and a test checks that each one is no more than 2048 bytes.

7.2 Bluetooth service (BLE-001, BLE-004, Q93, Q94, Q101, Q102, Q106, SR-04, SR-05, SR-07, SR-14, SR-16)

Item	Value
Service UUID	`bc32efc7-02d9-48cb-b66a-`

Item	Value
	`9041aa8cfa23`
To radio (phone writes, with response)	`85d5f69e-313a-458e-9912-20397554da2f`
From radio (radio notifies)	`4dee99ec-354d-451c-a12f-1a0a75cfbe6d`

Advertising (SR-14):

State	Address	Name	Contents
Normal running	Resolvable private address (LE privacy), changed at least every 15 minutes	None	Service UUID only
Pairing window open	Resolvable private address	“TrailBuddy” + short ID, so the rider can pick their radio	Service UUID, name
An encrypted phone connected	Not advertising	—	—

Paired phones recognise the radio through the identity key shared at pairing (Bluetooth Core Specification, Vol 3 Part C §10.7; Vol 6 Part B §1.3).

Framing (Q106):

- Each message is a 2-byte little-endian length followed by the encoded `Envelope`, split across as many Bluetooth packets as needed.
- A length of 0 or over 2048 is answered with ERROR TOO_LARGE. The receiver then discards every byte until 2 s pass with none arriving, and starts again with a new length. After 3 bad lengths in one minute, the radio drops the connection.
- A message not completed within 2 s of its last byte is discarded.
- The phone waits for each write to be confirmed before sending the next.

List sizes per part:

List	Entries per part
RIDERS_LIST	10
NEARBY_LIST	16
TRACK_POINTS	40 points (Q105)
GET_MISSED replays	One event per message

Every part carries the request ID; the last part sets `last`.

Connection order (Q102):

1. Connect.

2. HELLO (app) and WELCOME (radio). WELCOME carries the protocol version and pairing state only; nothing that identifies the radio (SR-14).
3. The phone starts encryption: it pairs if this is a new phone and the pairing window is open, or encrypts with the stored bond if not.
4. SESSION_START (app), carrying the phone's time; RADIO_INFO (radio), carrying the radio's identity.
5. Then the radio sends, in this order:
    1. any ERROR from 6.9
    2. RIDE_STATE
    3. SOS_STATUS, if its own SOS is active
    4. SOS_RECEIVED for every active SOS from others
    5. STATUS
6. Every other message.

Before encryption everything except HELLO and WELCOME is refused with NOT_ALLOWED_NOW, and the radio sends nothing but WELCOME, ERROR and GOODBYE.

Connections and time limits (SR-05):

- The radio accepts up to two Bluetooth connections at once, of which at most one may be encrypted. So a stranger's unencrypted connection can't take the only place.
- A connection that hasn't completed encryption within 5 s of connecting is dropped.
- A device address that has failed to encrypt is refused for 30 s.
- When a paired phone completes encryption, any other unencrypted connection is dropped.
- While an encrypted phone is connected, the radio stops advertising, so no third connection is attempted.
- The radio keeps advertising while it has no encrypted connection, so the rider's own phone can always reach it.

Pairing and security (SR-07):

- Bluetooth LE Secure Connections with bonding, Just Works, in Secure Connections Only mode with 128-bit keys (Bluetooth Core Specification, Vol 3 Part C §10.2.4). Legacy pairing is refused.
- A new phone can pair only while the pairing window is open (6.11).
- A phone that has already paired can reconnect at any time.
- Up to 4 phones are remembered. A fifth is refused, never pushed in by forgetting another. WELCOME tells the app the list is full, so it can tell the rider to hold the button for 10 s to forget all phones.
- The radio tells already-paired phones about new pairings: RADIO_INFO carries how many phones are paired, when the most recent one was added, and whether a new phone has paired since this phone last connected.

Keeping pairings, and the iPhone "forget this device" problem (BLE-004, Decision):

- An iPhone keeps its own copy of a pairing. If the radio loses its copy, the iPhone refuses to connect until the rider taps “Forget This Device” in the iPhone’s Bluetooth settings. Apps can’t delete that copy.
- So the radio never discards a pairing except on “forget phones” (button held 10 s) or factory reset. Pairings survive reboots, firmware updates and rollbacks.
- When the app sees the iPhone’s “pairing removed by the radio” error, it shows: “Go to Settings › Bluetooth, tap ⓘ next to your Trail Buddy radio, choose Forget This Device, then reconnect.”

Order of messages to the app (SR-16): the radio sends SOS_RECEIVED, SOS_STATUS, SOS_REPLY_RECEIVED and SOS_ENDED ahead of any other waiting message.

Session: there is no idle timeout once encrypted. After a disconnect, the radio carries on relaying, sending positions and presence, repeating its SOS and storing events. A radio in repeater mode accepts phones too.

7.3 Common rules and error codes (Q95, Q110)

- Every message carries a request ID (0 means unsolicited) and the `last` flag.
- Unknown fields are ignored.
- An unknown type, or a missing required field, is answered with ERROR.
- **Every value from the app is checked** against the same limits as over the air (2.6, 4.8): positions in range, heading 0–239 or 255, text and names valid UTF-8 with no control characters, sizes within the stated limits. A value outside them is BAD_FIELD.

Code	Meaning
UNKNOWN_TYPE	Message type not recognised
BAD_FIELD	Field missing, empty when it mustn’t be, out of range, invalid UTF-8, control characters, or a name over 24 bytes
NOT_ALLOWED_NOW	Not allowed in the current state (before HELLO, before encryption, or I’m OK for someone else’s alert)
VERSION_MISMATCH	Protocol version differs
NO_RIDE	Needs an active ride
RIDE_ACTIVE	A ride is already active; end it first
RIDE_EXPIRED	The ride code has expired (2.4)
SOS_ACTIVE	Not allowed during the rider’s own SOS
BUSY	Queue full; try later
TOO_LARGE	Text over 150 bytes, or a message over 2048 bytes
STORAGE_FAILED	The radio couldn’t save something
SETTINGS_RESET	Stored settings were damaged and reset to defaults
RIDE_RESET	The stored ride was damaged and erased

Code	Meaning
SOS_RESET	A stored active SOS was damaged and could not be resumed
LOW_BATTERY	Battery too low for this action
UPDATE_REJECTED	Firmware image refused
NO_CONFIRMATION	Text not confirmed by anyone after 3 resends

When several apply, the first in this order is returned: NOT_ALLOWED_NOW, VERSION_MISMATCH, UNKNOWN_TYPE, TOO_LARGE, BAD_FIELD, NO_RIDE, SOS_ACTIVE, RIDE_ACTIVE, RIDE_EXPIRED, LOW_BATTERY, BUSY, STORAGE_FAILED.

Examples:

Case	Code
A 151-byte text	TOO_LARGE
An empty text	BAD_FIELD
A 25-byte name	BAD_FIELD
A name containing a line feed	BAD_FIELD
JOIN_RIDE with an expiry already passed (clock known)	RIDE_EXPIRED
An update chunk at the wrong offset	BAD_FIELD, followed by UPDATE_PROGRESS giving the offset the radio expects

7.4 Message list

R = required, O = optional. Types follow Section 2.6. “Rider ID” on the phone link is the 3-byte rider tag. The phone link has 46 messages.

Connect

Message	Direction	Fields	Req
HELLO	App → Radio (before encryption)	protocol version (u32, R); app version (text ≤16, R)	BLE-001
WELCOME	Radio → App (before encryption)	protocol version (R); pairing window open (bool, R); phone list full (bool, R)	BLE-001, BLE-004
SESSION_START	App → Radio (first message after encryption)	phone time (time, R)	BLE-001
RADIO_INFO	Radio → App (reply to SESSION_START)	radio ID (8 bytes, R); short ID (text 3, R); firmware version (three u32: major,	BLE-001, DFU-001, SEC-003

Message	Direction	Fields	Req
		minor, patch, R); board model (enum: RAK19026_VC, RAK19007, POCKET_V3, T114_V2, THINKNODE_M1, R); last update result (enum: NONE, INSTALLED, ROLLED_BACK, O); phones paired (u8, R); newest pairing added (time, O); new phone paired since you last connected (bool, R)	
GOODBYE	Either	reason (enum: USER, SHUTDOWN, UPDATING, VERSION_MISMATCH, R)	BLE-001

- HELLO's phone time field from v0.4 is removed (field number reserved). Phone time now comes from SESSION_START (2.2).
- On a version mismatch, the radio replies with WELCOME, then GOODBYE with reason VERSION_MISMATCH, and disconnects.

Status

Message	Direction	Fields	Req
GET_STATUS	App → Radio	none	BLE-003
STATUS	Radio → App	battery (R); charging (bool, R); GPS (enum: NO_FIX, FIX, UNAVAILABLE, R); position source in use (enum: RADIO, PHONE, NONE, R); clock known (bool, R); clock conflict (bool, R); channel busy (R); slow-down step (u8 0–3, R); ride contact (u8, R); events waiting (u32, R); presence sending	BLE-003, MESH-004, NEAR-008, SEC-002

Message	Direction	Fields	Req
		(bool, R); storage failed (bool, R); nearby view incomplete (bool, R); frames sent as you (u32, R); firmware version (R)	

- **Ride contact:** the number of ride members in contact (6.3), not counting this radio.
- **STATUS is sent:**
  - in reply to GET_STATUS
  - on a change of charging, GPS state, clock conflict, ride contact, slow-down step, storage failed, nearby view incomplete or frames sent as you
  - when battery changes by 5 points or more
  - at most once every 30 s, except a change of storage failed or frames sent as you, which goes at once

Ride

Message	Direction	Fields	Req
JOIN_RIDE	App → Radio	ride name (text 1–24, R); ride key (16 bytes, R); tier (1–4, R); expiry (time, R)	RIDE-001, RIDE-002, RIDE-005
END_RIDE	App → Radio	none	RIDE-003
GET_RIDE	App → Radio	none	BLE-002
RIDE_STATE	Radio → App	ride active (bool, R); ride name (O); tier (O); expiry (time, O); key check value (4 bytes, O); own rider tag (O); over rider limit (bool, O); error code (O)	RIDE-001, RIDE-004, RIDE-005, BLE-002

JOIN_RIDE rules:

- Refused with RIDE_ACTIVE if a ride is active.
- Refused with SOS_ACTIVE during the rider's own SOS.
- Refused with BAD_FIELD for an all-zero key, a wrong key length, a tier outside 1–4, an expiry of 0, or an expiry more than 8 days after a known clock.
- Refused with RIDE_EXPIRED if the clock is known and the expiry has passed.
- The radio draws a new join ID, sets the frame counter ceiling, and saves the ride in one all-or-nothing write that survives reboot.

- RIDE_STATE replies to JOIN_RIDE, END_RIDE and GET_RIDE, reading back what was stored, and is sent unprompted when a ride expires. The key itself is never sent back; the check value proves it.

Messaging

Message	Direction	Fields	Req
SEND_TEXT	App → Radio	text (1–150 bytes, R)	MSG-001
SEND_RESULT	Radio → App	message ID (u16, R); state (enum: QUEUED, SENT, DELIVERED, FAILED, R); confirmations (u8, O); in contact when sent (u8, O); error code (O)	MSG-002
TEXT_RECEIVED	Radio → App	event number (u32, R); message ID (u16, R); sender rider ID (R); text (R); sent (time, R); time received (time, R); old (bool, R); time suspect (bool, R); RSSI and SNR (R)	MSG-001, SEC-004
GET_MISSED	App → Radio	after event number (u32, R)	MSG-001
EVENTS_INFO	Radio → App	oldest event number held (u32, R); newest event number (u32, R)	MSG-001

Location

Message	Direction	Fields	Req
SHARE_MY_LOCATION	App → Radio	latitude, longitude (R); accuracy (R); fix time (R); fix age when sent (u8 seconds, R); heading (O); speed (O)	POS-001, POS-002
LOCATION_UPDATE	Radio → App	rider ID (R); latitude, longitude (R); accuracy (R); fix time (R); source (R); hops travelled (u8, R); time suspect	POS-001, POS-002

Message	Direction	Fields	Req
		(bool, R)	

LOCATION_UPDATE rules (Q91):

- Each new authenticated position from a ride member is reported once.
- A position older, by fix time, than one already held for that rider is stored in history but not reported.
- **Hops travelled:** the kind's starting hops minus hops remaining at receipt.
- **Own position:** the radio also reports the position it sends, with its own rider tag, when the source is radio GPS.

Fill on Return

Message	Direction	Fields	Req
GET_TRACK	App → Radio	rider ID (R); from time (R); to time (O, default now)	FILL-001
TRACK_POINTS	Radio → App	rider ID (R); points (list of latitude, longitude, fix time; at most 40 per part, sorted by fix time, R); filled (bool, R); outline (bool, R)	FILL-001

SOS

Message	Direction	Fields	Req
SEND_SOS	App → Radio	latitude, longitude (O); accuracy (O); fix time (O). If absent, the radio uses 6.4.	SOS-001, SOS-002
SOS_STATUS	Radio → App	alert ID (R); state (enum: ACTIVE, ENDED_OK, ENDED_LEFT_RIDE, R); times sent (u16, R); last sent (time, R); position sent (latitude, longitude, fix time, or no-position flag, R); copy-that riders (list of rider IDs, R)	SOS-003
SOS_RECEIVED	Radio → App	event number (R); alert ID (R); rider ID	SOS-001, SOS-005

Message	Direction	Fields	Req
		(R); latitude, longitude, accuracy, fix time, or no-position flag (R); first sent (time, R); repeat number (u16, R); time suspect (bool, R)	
SOS_REPLY	App → Radio	alert ID (R); alert owner rider ID (R); reply (enum: COPY_THAT, IM_OK, R)	SOS-004
SOS_REPLY_RECEIVED	Radio → App	event number (R); alert ID (R); alert owner rider ID (R); from rider ID (R); reply (R); old (bool, R)	SOS-004
SOS_ENDED	Radio → App	event number (R); alert ID (R); alert owner rider ID (R); reason (enum: IM_OK, LEFT_RIDE, R)	SOS-003, SOS-004, SEC-002
GET_SOS	App → Radio	none	SOS-003

The owner secret is never sent over the phone link.

Riders

Message	Direction	Fields	Req
GET_RIDERS	App → Radio	none	MSG-003
RIDERS_LIST	Radio → App	list (at most 30, 10 per part) of: rider ID; radio ID (O until a hello is heard); short ID (O); display name (O); repeater (bool); last heard (seconds ago, u32); RSSI and SNR (O: direct receptions only); in contact (bool)	MSG-003

Message	Direction	Fields	Req
RIDER_UPDATE	Radio → App	event number (R) plus the fields of one RIDERS_LIST entry	MSG-003

RIDER_UPDATE is sent at most once per 10 s per rider, when:

- a rider is first heard
- a name changes
- a rider's tag changes (the entry moves to the new tag)
- contact is lost or regained (6.3)

Riders are never removed during a ride.

Nearby

Message	Direction	Fields	Req
NEARBY_RIDER	Radio → App	anonymous ID (R); latitude, longitude (R); heading (R); speed (R); fix age at receipt (R); received (time, R); RSSI and SNR (R)	NEAR-004
GET_NEARBY	App → Radio	none	NEAR-004
NEARBY_LIST	Radio → App	list (at most 32, 16 per part) of NEARBY_RIDER entries not yet expired; view incomplete (bool, R)	NEAR-004, NEAR-008

Settings

Message	Direction	Fields	Req
GET_SETTINGS	App → Radio	none	BLE-002
SETTINGS	Radio → App	all settings, as stored	BLE-002
CHANGE_SETTINGS	App → Radio	list of setting ID and new value	BLE-002
SETTINGS_RESULT	Radio → App	OK or error code (R); all settings as now stored (R)	BLE-002

ID	Setting	Type	Default
1	Display name	text 1–24	"Rider" + short ID

ID	Setting	Type	Default
2	Presence sending	bool	on
3	Repeater mode	bool	off
4	Removed in v0.4 (was GPS source preference). Never reused.	—	—

Updates

Message	Direction	Fields	Req
START_UPDATE	App → Radio	new version (three u32, R); board model (R); image size (u32, R); image hash (32 bytes SHA-256 of the whole image, R); signature (64 bytes, R)	DFU-001
UPDATE_CHUNK	App → Radio	offset (u32, R); data (≤200 bytes, R)	DFU-001
UPDATE_PROGRESS	Radio → App	bytes received (u32, R); state (enum: RECEIVING, VERIFYING, INSTALLING, REJECTED, ABORTED, R); error code (O)	DFU-001
ABORT_UPDATE	App → Radio	none	DFU-001

Update rules (DFU-001, Q97, Q109, SR-08, SR-12):

- **Routes (Decision):** updates install over Bluetooth (these messages) or over USB through the signature-checking bootloader's own USB update mode. Both routes accept only signed images. **Shipped radios have no unsigned route:** the drag-and-drop (UF2) bootloader is replaced by the signature-checking bootloader before a radio is shipped. Development radios may keep the UF2 bootloader; they are never shipped and never hold a production signing key.
- **What is signed (SR-12):** the signature is pure Ed25519 as defined in RFC 8032 §5.1 (not Ed25519ph or Ed25519ctx), over these 65 bytes:

Bytes	Content
16	ASCII `trailbuddy v1 fw`
12	new version: major, minor, patch, each u32

Bytes	Content
	little-endian
1	board model, as its enum number (RAK19026_VC = 1, RAK19007 = 2, POCKET_V3 = 3, T114_V2 = 4, THINKNODE_M1 = 5)
4	image size, u32 little-endian
32	SHA-256 of the whole image

The version, board and size checked by the radio are the signed ones.

- **Who checks:** the bootloader itself verifies the signature, with a public key stored where application firmware can't change it, both when installing and before every start-up. An image that fails is never run, whichever route delivered it.
- **Signing keys:** the bootloader holds two public keys, a primary and a backup. The product owner keeps both private keys outside the repository, the backup offline. If the primary private key is compromised, the product owner issues an update signed with the backup key, after which the bootloader accepts only the backup key; that change can't be undone. If the chosen bootloader can't do this, the clean team raises it in OPEN-QUESTIONS.md.
- **Refused with UPDATE_REJECTED:**
  - a wrong board model
  - a bad signature or hash
  - a version equal to or older than the installed one
  - an image larger than the update slot
- **Refused with LOW_BATTERY:** battery under 40%.
- **Refused with SOS_ACTIVE:** during the rider's own SOS.
- **Refused with NOT_ALLOWED_NOW:** START_UPDATE while an update is already in progress.
- A chunk at the wrong offset gets BAD_FIELD and an UPDATE_PROGRESS with the expected offset.
- With no chunk for 60 s, the update is aborted (state ABORTED).
- **While receiving,** the radio keeps relaying and repeating any SOS. SEND_SOS during an update aborts the update and starts the SOS.
- **After VERIFYING**, the radio reboots into the new image. It is offline for at most 90 s.
- **Confirming:** the new firmware confirms itself only after its radio, Bluetooth and flash checks pass, within 60 s of starting. Otherwise the bootloader rolls back, only to the previously confirmed image.

- The result reaches the app in the next RADIO_INFO.
- Pairings, the ride record and counters keep their format and survive every update and rollback (4.4, 7.2).
- The clean team records the bootloader chosen for each board, how its USB mode is entered, and the flash layout in DECISIONS.md.

Repeater, errors and diagnostics

Message	Direction	Fields	Req
REPEATER_STATUS	Radio → App	frames relayed since boot (u32); uptime (seconds, u32); battery	RPT-001
ERROR	Radio → App	error code (R)	BLE-001
GET_DIAGNOSTICS	App → Radio	none	DIAG-001
DIAGNOSTICS	Radio → App	see below	DIAG-001
CLEAR_DIAGNOSTICS	App → Radio	none; the reply is DIAGNOSTICS with every counter at zero	DIAG-001

DIAGNOSTICS counters (Q96, Q107), all u32, wrapping to 0. They start at zero at boot or on clear; End Ride doesn't clear them.

- **For each of the 9 ride kinds** (counted only once the kind is known, after authentication):
  - **sent:** own frames transmitted
  - **received:** frames that passed authentication
  - **relayed:** relays actually transmitted
  - **suppressed:** relays cancelled by suppression
  - **gave up:** frames dropped after 8 busy checks
- **For presence:** sent, received (passing checks 1–3), gave up.
- **Drops by reason,** each frame counted once, under the first check it fails in 4.8: unknown version, malformed, wrong ride, failed authentication, sent as you, duplicate. Plus queue full (4.5).
- **Security and time:** time suspect, bad SOS proof, SOS over schedule, nearby overflow.
- **Channel busy history:** the last 60 complete minutes, oldest first. The minute in progress isn't included.

8. Ride QR code (RIDE-001, RIDE-005, Q98, SR-11)

Text form: `trailbuddy://ride/` followed by the bytes below in Base64url (RFC 4648 §5): no padding, only the characters A–Z, a–z, 0–9, "-" and "_", with letter case significant.

Offset	Size	Field
0	1	Format version (2)
1	1	Tier (1–4)
2	16	Ride key
18	4	Expiry (Trail Buddy time, u32 little-endian): the creation time plus 7 days
22	1	Ride name length (1–24)
23	n	Ride name (UTF-8)
23 + n	2	CRC, low byte first

The CRC is CRC-16 with polynomial 0x1021, start value 0xFFFF, no bit reflection and no final XOR. This is known as CRC-16/CCITT-FALSE; the parameters here are complete, so no other document is needed. It covers every byte before it.

The maximum is 49 bytes, or 66 characters. The app refuses a code with:

- a format version other than 2 (version 1 codes have no expiry and are no longer accepted)
- a bad CRC
- a wrong length for its name
- bytes after the CRC
- any character outside the set above
- an expiry that has passed

9. Hardware (HW-001, Q99, Q113, Q114, Q115)

Version 1 boards (Decision). All use a Nordic nRF52840 and a Semtech SX1262, and one firmware serves them all, with a board profile for each.

Board	Role	Starting point	Profile source in `docs/hardware/`
RAKwireless RAK19026 VC base board with RAK4631 core	Lead board: every build step runs here first	Zephyr's RAK4631 board definition	RAK19026 VC datasheet; RAK4631 documentation
Heltec Mesh Node T114 V2	Version 1 board	Zephyr's Heltec T114 V2 board definition	Heltec T114 datasheet (cleaned), T114 V2.0 schematic, Zephyr T114 V2 board page (cleaned)
Elecrow ThinkNode M1	Version 1 board	No Zephyr definition; the clean team writes one	ThinkNode M1 datasheet (cleaned); the full schematic if

Board	Role	Starting point	Profile source in `docs/hardware/`
			Elecrow supplies it

Zephyr board definitions are part of the open-source Zephyr project, not another mesh project's board files. Manufacturer documents go into `docs/hardware/` only as copies cleaned by the product owner, because some mention other firmware. **Blacked-out text** in those copies described another firmware's use of buttons and lights, not the hardware; it isn't needed and the clean team ignores it (Q114).

What version 1 uses on each board. The clean team confirms every item, pin and polarity from the manufacturer documents and records it in the board profile. Anything a document doesn't confirm is recorded as unknown in the profile and raised in OPEN-QUESTIONS.md.

Role	RAK19026 VC	T114 V2	ThinkNode M1
User button (6.11)	SW1	User button (USER_SW, P1.10 on the schematic)	Function button (FK, P1.07)
GNSS	Integrated	GPS module on the 8-pin GPS header, fitted on every Trail Buddy T114	Integrated Quectel L76K, with a GPS on/off switch
Status light (9)	LED 1	Green LED (P1.03 on the schematic)	Blue status LED (LED3, P0.13)
Battery measurement	From the RAK documentation	BAT_ADC through the switched divider shown on the schematic	Not documented; reported as 255 (unknown) until Elecrow supplies it
Antenna	Gain from the supplied antenna's specification	Gain from the supplied antenna's specification	2.5 dBi (datasheet)
Not used in version 1	LED 2	RGB LEDs, display	Buzzer, e-paper display and front light, page button

- **GNSS switched off or failed:** if the ThinkNode M1's GPS switch is off, or any board's GNSS doesn't respond, STATUS reports GPS UNAVAILABLE and the radio uses phone positions (6.1, 6.6). The M1's switch has no processor pin, so the radio sees only that the module doesn't respond. It never fails because GNSS is missing.
- **Power and reset controls** (reset buttons, the ThinkNode M1's power switch, the RAK19026's battery switch) keep their hardware function. "Starting" in 6.11 means power-on or reset by whatever means the board provides; each profile records how. On the ThinkNode M1, starting means turning the power knob on, so a factory reset is holding the function button while turning the knob on (Q114).

- **The radio never sleeps** (2.2, Q113). POLICY §18's hardware tests cover processor idle and wake, and GPS duty cycling (6.1), not a sleep mode.
- **Status light (Decision):** each board's status light shows only power, mode and pairing:

State	Status light
Firmware starting (power-on or reboot)	On steadily for 2 s
Running normally	One short blink (50 ms) every 5 s, so the rider can see the radio is on
Pairing window open (6.11)	Two short blinks every 2 s
New phone paired	Three quick blinks, once
Rebooting to install an update (7.4, Updates)	Fast blink (5 times a second) until the new firmware starts
Factory reset under way (6.11)	On steadily until the reset finishes
Off, or battery flat	Off

  A development radio's UF2 bootloader drives the light in its own way. Lights wired to the charger in hardware keep their own function.
- **Buzzers and screens** are not used in version 1. The firmware doesn't drive them.
- **Unknowns for the product owner (Q114, Q115):**
  - ThinkNode M1: whether version 1 uses the casing version; the full Elecrow schematic (battery measurement, TCXO voltage, RF switch polarity, the P0.21/DIO3 listing); the L76K manual.
  - T114 V2: the GPS module's model and manual (serial speed, which side "TX_GPS" refers to).
  - Until these arrive, the clean team records each as unknown in the profile; they block only that board's build step 12.
- **The WisMesh Pocket V3** gets its own profile once the board and its documentation are available.

10. Diagnostics

- **DIAG-001:** the radio keeps the counters in 7.4 in release firmware, for field testing.
- Detailed logging is removed from release builds. No log or counter ever holds key material (4.9).

11. Test vectors and sample values (Q56, Q73)

These vectors are exact. Implementations must reproduce them byte for byte. The security review independently recomputed every v0.4 vector; the v0.5 vectors below replace them.

11.1 Key derivation and IDs

Input or output	Hex
Ride key	`000102030405060708090a0b0c0d`

Input or output	Hex
	`0e0f`
Radio ID	`1122334455667788`
Join ID	`0a0b0c0d`
Encryption key	`91623a7d358eb9fae7a1497546128f3c`
Ride tag	`20bb`
Rider tag, clash counter 0	`eea021`
Rider tag, clash counter 1	`c4b67b`
Key check value	`715305b1`
Short ID	`CER`
SOS owner secret	`202122232425262728292a2b2c2d2e2f`
SOS owner commitment	`0ee99c244d1b9886`

11.2 Ride frames

All use the key, radio ID and join ID above, clash counter 0 and, unless stated, whole-ride destination. The position is latitude 35.349120°, longitude −120.247385°, accuracy 6 m, fix time 25,000,000.

Frame	Inputs	Complete frame (hex)
Position	counter 42, hops 3, class 2, source radio GPS	`113220bbeea0210a0b0c0d2a000000000000094d0576b3c22a1dc3e65137bca408cfafe70891487641`
Same position, relayed once	identical except hops 2	`112220bbeea0210a0b0c0d2a000000000000094d0576b3c22a1dc3e65137bca408cfafe70891487641`
Hello	counter 43, hops 3, class 4; radio ID above, anonymous ID a1b2c3d4, flags 0, name “Dave”	`113420bbeea0210a0b0c0d2b0000000000004a9d64a0081af2f09916d030aafc326e31700dab331ce6bccdc174`
Hello request	counter 44, hops 3, class 4	`113420bbeea0210a0b0c0d2c000000000000a1f24a057d38b05300`
Text	counter 45, hops 3, class 5; message ID 7, send time 25,000,050, “Waiting at the fork”	`113520bbeea0210a0b0c0d2d000000000000d6cf80517c1f473171772a775708113bc2db951f67ef8aa504545f71bf7ef44f9f88d6`

Frame	Inputs	Complete frame (hex)
Delivered	counter 46, hops 3, class 3, destination c4b67b; message ID 7, original sender c4b67b	`113320bbeea0210a0b0c0d2e000000c4b67b528a84fbdf9a56108e524e3c6bf0`
SOS	counter 47, hops 7, class 0; alert ID 3, the position above, first sent 25,000,100, repeat 0, commitment from 11.1	`117020bbeea0210a0b0c0d2f00000000000010a821295b9c92436442d99245294feb9eed72c5ed3d4d4555fb188baacaa521931f759763cd`
SOS, no position	counter 48, hops 7, class 0; alert ID 4, latitude and longitude 0x7FFFFFFF, accuracy 255, fix time 0, first sent 25,000,200, repeat 0, same commitment	`117020bbeea0210a0b0c0d300000000000000b50b33e9e73ca73bb02f7b3ea0af647436d19eac0a7c215aa15ba14e7439fd5983be8aab4ec5`
Copy that	counter 49, hops 7, class 0; alert ID 3, owner eea021, reply 1, send time 25,000,120	`117020bbeea0210a0b0c0d310000000000000bd07fef3d53d5591c599982b4c192f04c90a99`
I'm OK	counter 50, hops 7, class 0; alert ID 3, owner eea021, reply 2, send time 25,000,400, owner secret from 11.1	`117020bbeea0210a0b0c0d32000000000000474dc68ac09cd1db0551912d4e6ecf30555c554ed883a031af3e081776926e96974b64`
Fill ask	counter 51, hops 3, class 6; 1 rider: tag 5a6b7c, from 24,998,200 to 25,000,000	`113620bbeea0210a0b0c0d330000000000006539e3bce2901cd05a8f3e5a64b378ea3391022b3b`
Fill answer	counter 52, hops 3, class 6; own tag eea021, range 24,998,200 to 25,000,000, outline part 0 of 1, 3 points: the position above plus i × 50 to latitude and minus i × 50 to longitude, fix time 24,998,200 + i × 60, for i = 0, 1, 2	`113620bbeea0210a0b0c0d34000000000000053415110097820 88a5001473 6e21878201a88fa69e25249ed37591143db3a5283dec5680e07cb8d9fd64d690bc69da7d76e68b790ddf9f6b9533f5`
Fill answer, no history	counter 53, hops 3, class 6; own tag eea021, range as above, part byte 0x00, total 1, point count 0	`113620bbeea0210a0b0c0d350000000000000 86bffba13011e6305ead459d471cbb3b88ab020e37d091`

11.3 Frames that must be rejected

Case	Frame (hex)	Expected result
Tampered authentication tag (the position frame with its last byte changed)	`113220bbeea0210a0b0c0d2a000000000000094d0576b3c22a1dc3e65137bca408cfafe70891487640`	Dropped, “failed authentication” (check 6)
Hops above the start value (the position frame with hops 4 in a tier 1 ride)	`114220bbeea0210a0b0c0d2a000000000000094d0576b3c22a1dc3e65137bca408cfafe70891487641`	Passes authentication (hops aren’t authenticated), then dropped as malformed (check 12)
Replayed counter (the position frame received a second time)	The position frame from 11.2	Dropped, “duplicate” (check 8)
Class that doesn’t match the kind (a valid position encrypted with class 0, counter 54)	`113020bbeea0210a0b0c0d360000000000000902ba5e7004e373150184eab531cc9ca2501654d3c3c26`	Passes authentication, then dropped as malformed (check 10)
Sent as you (any frame above, received by the radio whose rider tag is eea021 and join ID 0a0b0c0d, that it didn’t send)	—	Dropped, “sent as you” (check 7); STATUS frames sent as you + 1
I’m OK with a wrong secret (the I’m OK frame rebuilt with any other 16-byte secret)	—	Authenticates, but the SOS isn’t ended; counted “bad SOS proof”

11.4 Presence frame

Inputs:

- anonymous ID a1b2c3d4
- the position above, rounded to 35.3491° and −120.2474° (sent as 35,349,100 and −120,247,400)
- heading 90° (value 60)
- speed 32 km/h
- fix age 2 s

Frame: `12a1b2c3d46c621b02982bd5f83c2002` (unchanged from v0.4)

11.5 Ride QR code

Inputs: tier 1, the ride key above, expiry 25,604,800 (fix time 25,000,000 plus 7 days), name “Sat Dez Loop”. The CRC is 0xB19F, sent low byte first as `9fb1`.

Item	Value
Bytes (37)	`02010001020304050607080 90a0b0c0d0e0fc0b286010c5361742044657a204c6f6f709fb1`
Text	`trailbuddy://ride/AgEAAQIDBAUGBwgJCgsMDQ4PwLKGAQxTYXQgRGV6IExvb3CfsQ`

11.6 Firmware signature (SR-12)

A test key only, from RFC 8032 §7.1 test 1. Never use it to sign real firmware.

Item	Value
Private key (test only)	`9d61b19deffd5a60ba844af492ec2cc44449c5697b326919703bac031cae7f60`
Public key	`d75a980182b10ab7d54bfed3c964073a0ee172f3daa62325af021a68f707511a`
Image	200 bytes, every byte 0x33
Image SHA-256	`eaa90984064eb28c94c70cabc2f3a68e807613fb4d438f75482b543850f0d0dd`
Signed bytes (65): label, version 1.0.1, board RAK19026_VC (1), size 200, hash	`747261696c6275646479207631206677010000000000000010000000 1c8000000eaa90984064eb28c94c70cabc2f3a68e807613fb4d438f75482b543850f0d0dd`
Signature	`aa37ea0c6cc6859b6538fbff7a2366d3b78b8096f969a2aaf72094dad7c0e002ecdc484157666c5063fd7877acde504079dea35fffb89274c186080003a28f02`

11.7 Phone-link samples (Q55)

The clean team encodes each sample once with the generated code and saves the input and its bytes together in `spec/vectors/`. Those saved bytes then become the reference for both the firmware and app tests. The spec team checks the decoded values against these samples.

Message	Sample
HELLO	protocol version 1; app version “1.0.0”
WELCOME	version 1; pairing window open false; phone list full false
SESSION_START	phone time 25,000,000
RADIO_INFO	radio ID 1122334455667788; short ID “CER”; firmware 1.0.0; RAK19026_VC; last update NONE; phones paired 1; newest pairing 24,990,000; new phone paired since you last connected false
GOODBYE	USER

Message	Sample
GET_STATUS	(no fields)
STATUS	battery 84; not charging; FIX; RADIO; clock known; no clock conflict; busy 14; step 0; contact 3; events waiting 2; presence on; storage OK; nearby view complete; frames sent as you 0; firmware 1.0.0
JOIN_RIDE	“Sat Dez Loop”; key as 11.1; tier 1; expiry 25,604,800
END_RIDE	(no fields)
GET_RIDE	(no fields)
RIDE_STATE	active; “Sat Dez Loop”; tier 1; expiry 25,604,800; check value 715305b1; rider tag eea021; not over limit
SEND_TEXT	“Waiting at the fork”
SEND_RESULT	message ID 7; DELIVERED; 3 confirmations; 3 in contact when sent
TEXT_RECEIVED	event 101; message ID 7; sender c4b67b; “Waiting at the fork”; sent 25,000,048; received 25,000,050; not old; not time suspect; RSSI −92; SNR 26 (6.5 dB)
GET_MISSED	after event 100
EVENTS_INFO	oldest 1; newest 105
SHARE_MY_LOCATION	35.349120°, −120.247385°; accuracy 6 m; fix time 25,000,000; fix age 1 s; heading 90°; speed 32 km/h
LOCATION_UPDATE	rider c4b67b; 35.349170°, −120.247435°; accuracy 8 m; fix time 25,000,010; RADIO; hops travelled 1; not time suspect
GET_TRACK	rider c4b67b; from 24,998,200; to 25,000,000
TRACK_POINTS	rider eea021; the three points of the fill answer in 11.2; filled; outline
SEND_SOS	position as SHARE_MY_LOCATION
SOS_STATUS	alert 3; ACTIVE; times sent 2; last sent 25,000,110; position as above; copy-that riders [c4b67b]
SOS_RECEIVED	event 102; alert 3; rider eea021; position as above; first sent 25,000,100; repeat 1; not time suspect
SOS_REPLY	alert ID 3; owner eea021; COPY_THAT

Message	Sample
SOS_REPLY_RECEIVED	event 103; alert 3; owner eea021; from c4b67b; COPY_THAT; not old
SOS_ENDED	event 104; alert 3; owner eea021; IM_OK
GET_SOS	(no fields)
GET_RIDERS	(no fields)
RIDERS_LIST	one entry: c4b67b; radio ID 8877665544332211; short ID from it; “Dave”; not repeater; last heard 4 s; RSSI −92; SNR 26; in contact; last part
RIDER_UPDATE	event 105 plus the entry above
NEARBY_RIDER	anonymous ID a1b2c3d4; 35.3491°, −120.2474°; heading 90°; speed 32 km/h; fix age 2 s; received 25,000,020; RSSI −101; SNR 8 (2 dB)
GET_NEARBY	(no fields)
NEARBY_LIST	one entry as NEARBY_RIDER; view complete; last part
GET_SETTINGS	(no fields)
SETTINGS	1 = “Dave”; 2 = on; 3 = off
CHANGE_SETTINGS	1 = “Dave”; 2 = on
SETTINGS_RESULT	OK; 1 = “Dave”; 2 = on; 3 = off
START_UPDATE	version 1.0.1; RAK19026_VC; size 200; hash and signature from 11.6
UPDATE_CHUNK	offset 0; 200 bytes of 0x33
UPDATE_PROGRESS	200 bytes received; RECEIVING
ABORT_UPDATE	(no fields)
REPEATER_STATUS	relayed 1,234; uptime 3,600 s; battery 77
ERROR	request ID 12; BAD_FIELD
GET_DIAGNOSTICS	(no fields)
DIAGNOSTICS	every counter 0 except position sent 5, received 12, relayed 3; busy history 60 × 14
CLEAR_DIAGNOSTICS	(no fields)

11.8 Review

The security review of v0.4 (SR-01 to SR-21) is answered in this version (Section 15). The reviewer confirms the v0.5 answers and recomputes the v0.5 vectors before freeze.

12. Requirement and policy changes in this version

Issued in REQUIREMENTS v0.5 and POLICY v0.5. Each has its own change table.

- **REQUIREMENTS:**
  - SOS-003: repeated at least every 30 seconds.
  - SOS-004: only the owner can end an SOS, proven by the owner secret.
  - SOS-006 (new): SOS delivery target (4.7).
  - MESH-005: listen before talking, with SOS channel wait bounded to 1 s.
  - RIDE-005 (new): new key every ride; codes expire after 7 days.
  - NEAR-002 and NEAR-009 (new): what presence reveals, told to the rider.
  - BLE-004 (new): pairing rules and the iPhone “forget this device” handling.
  - DFU-001: signed updates over Bluetooth and USB; no unsigned route on shipped radios.
  - SEC-001 to SEC-005 (new): nonce uniqueness, SOS integrity, pairing control, replay and time handling, key handling.
  - NFR-001: frames up to 184 bytes; fill target applies below 15% busy.
  - Cited for the first time in the spec (Q111): MESH-005 (4.7), SOS-005 (6.4, SOS_RECEIVED), HW-001 (9). The v0.4 edits to MESH-003, MESH-004, RPT-001 and NEAR-005 are now listed in REQUIREMENTS’ change table.
- **POLICY:**
  - §8: board unknowns, antenna gains, no sleep mode.
  - §12: threat model, debug port locked, key handling, signed updates on both routes, bootloader checks before every start-up.
  - §15: library licences, including the Nordic licence on the crypto libraries; the signature-checking bootloader replaces UF2 on shipped radios.
  - §17 and §18: simulator scenarios for bursts, SOS delivery and SOS floods; fuzz tests.
  - §20: done-criteria for the above.

13. Approved standards (Q63)

The spec team approves these as sources. The product owner adds the official documents to `docs/standards/` and logs each in SOURCE-REGISTER.md; sessions never fetch them from the web.

Standard	Defining document
HKDF	RFC 5869
SHA-256	FIPS 180-4
AES	FIPS 197
CCM mode	NIST SP 800-38C and RFC 3610
Ed25519	RFC 8032 (§5.1 and the §7.1 test vectors)
Base64url, Base32 alphabet	RFC 4648 (§5 and §6)

Standard	Defining document
UTF-8	RFC 3629
Protocol Buffers	Google's proto3 language specification
UTC	ITU-R TF.460
GPS time	IS-GPS-200
QR code	ISO/IEC 18004. For the app team only; the firmware never makes or reads QR codes.
CRC-16/CCITT-FALSE	No document; fully defined in Section 8
Bluetooth LE: pairing, Secure Connections Only mode, LE privacy, 128-bit UUIDs	Bluetooth Core Specification, including Vol 3 Part C §10.2.4 and §10.7, Vol 3 Part H, Vol 6 Part B §1.3
FCC rules	47 CFR Part 15, §15.247
UF2	Microsoft's published UF2 format, with the board's UF2 bootloader documentation (development radios only)

14. Still open

- ☐ Product owner decision: **SOS without a phone.** Today the radio can't raise an SOS on its own. Options include a deliberate button pattern on the radio. Undecided.
- ☐ Compliance lab: confirm the settings and regulatory route in 2.1, with each board's supplied antenna.
- ☐ Mesh simulator: confirm the airtime figures in 2.5, the slow-down steps in 6.10, the SOS delivery target in 4.7, burst behaviour (Q104) and SOS-flood behaviour (4.5).
- ☐ Security reviewer: confirm the v0.5 answers to SR-01 to SR-21 and recompute the v0.5 vectors.
- ☐ Clean team: confirm on the device, in build step 2, that the chosen crypto library handles AES-128-CCM with a 13-byte nonce and 8-byte tag, HKDF-SHA256, SHA-256 and Ed25519; pin the SDK version; record each library's licence.
- ☐ Clean team: confirm the chosen bootloader can check signatures before every start-up, take signed updates over USB, and switch from the primary to the backup key (7.4).
- ☐ Product owner: add the standards in Section 13 to `docs/standards/`.
- ☐ Product owner: antenna specifications for the RAK19026 VC and T114 V2 (2.1, 9).
- ☐ Product owner: ThinkNode M1 casing version, full schematic and L76K manual; T114 V2 GPS module model and manual (9).
- ☐ Product owner: the WisMesh Pocket V3 documents, when the board ships.
- ☐ "Approaching" definition: deferred by the product owner.

15. Where each question and finding is answered

Questions and findings	Section
Q2	3
Q5, Q92, Q105	7.1, 7.2
Q7, Q71, Q72, Q73	2.3, 4.4, 6.11

Questions and findings	Section
Q15, Q59, Q69	4.5
Q25	6.3, 6.10, 7.4 Status
Q38, Q58, Q84, Q108	6.5
Q39, Q81, Q82	6.4
Q47	6.6, 7.2
Q54	9
Q55	11.7
Q56	11.1 to 11.6
Q57, Q77, Q116	6.1, 6.6, 7.4 Location
Q60, Q103	6.3
Q61	3, 7.4 Settings
Q62	0, 2.6, 12, and REQUIREMENTS and POLICY v0.5
Q63	13
Q64	4.7
Q65, Q66	6.10, 2.5
Q67	4.8
Q68	4.6
Q70, Q100	4.4
Q74	5, 6.3
Q75	2.2
Q76, Q78	6.6
Q79, Q80, Q83	3, 6.4
Q85	6.12
Q86	6.3
Q87, Q88, Q110	6.9, 7.3
Q89	6.8
Q90, Q112	6.4, 9
Q91	7.4 Location
Q93, Q94, Q101, Q102, Q106	7.2, 6.11
Q95	7.3
Q96, Q107	7.4 Diagnostics
Q97, Q109	7.4 Updates
Q98	8
Q99	2.1, 9

Questions and findings	Section
Q104	2.5, 6.2, 6.3, 6.5
Q111	12, and requirement IDs in section titles and message tables
Q113	2.2, 9
Q114, Q115	9
SR-01	2.3, 4.1, 4.3, 4.4
SR-02	4.4, 6.12
SR-03	4.8, 5, 6.4
SR-04	2.2, 4.8, 7.2
SR-05	7.2
SR-06	4.5, 4.6, 4.8
SR-07	6.8, 6.11, 7.2, 9
SR-08	7.4 Updates
SR-09	4.6, 4.7, 6.4
SR-10	2.3, 5, 6.4
SR-11	2.4, 4.4, 8
SR-12	7.4 Updates, 11.6
SR-13	4.8, 7.2, 7.3, POLICY §18
SR-14	2.3, 7.2, 7.4 Connect
SR-15	2.3, 2.7, 6.6
SR-16	6.6, 7.2
SR-17	4.6, 6.10
SR-18	2.4, 4.9, POLICY §12
SR-19	2.3, 6.2
SR-20	6.3
SR-21	14, POLICY §15

16. Changes from v0.4

- **Frames:** the envelope gains a 4-byte join ID (18 bytes; overhead 26; frames up to 184 bytes). The nonce is rider tag, join ID, counter and ride tag. Every vector is new.
- **Counter:** now per join, stored with the ride record; read-back, highest-copy, missing-is-damaged, fixed-format and never-wraps rules; SOS still goes out if storage fails; flash wear budget.
- **SOS:**
  - repeats every 30 s at most (was 5 minutes)
  - only the owner can end it, proven by the owner secret; “I’m OK” sent twice and re-sent when needed
  - repeat number is 2 bytes

  - own SOS has reserved queue space; relayed SOS traffic is limited to the SOS schedule
  - relay window 20–500 ms, listen before talking with a 1 s bound, hops can only grow
  - a delivery target for the simulator
  - frames are never dropped because of their time
- **Rides:** a new key every ride; QR format version 2 with a 7-day expiry; RIDE_EXPIRED.
- **Clock:** phone time only from a paired phone after encryption, with a 5-minute step limit.
- **Receive checks:** maximum length before decryption, reserved sender tag, “sent as you”, class matches kind, enumerated values, the general “anything not listed is invalid” rule.
- **Phone link:** 46 messages (SESSION_START and RADIO_INFO added; WELCOME no longer identifies the radio); advertising with private addresses; two connection places and a 5 s encryption deadline; framing limit 2048 bytes with re-synchronisation; TRACK_POINTS 40 per part; fixed-width protobuf types; SOS messages first; new STATUS and message fields.
- **Pairing:** window only on a button press or at first set-up; one new phone per window; fifth phone refused; paired phones told of new pairings; pairings kept through updates; iPhone “forget this device” handling; status light shows pairing.
- **Updates:** signed over Bluetooth and USB; no unsigned route on shipped radios; version, board and size signed; bootloader checks before every start-up; primary and backup keys; a real signature vector.
- **Other:** threat model (2.7); key handling (4.9); presence coordinate unit stated; nearby table can’t be flooded; Delivered count and resend rules; burst spreading; empty fill answer; counters only after authentication; no sleep mode; antenna limit 3 dBi; board unknowns listed.